ЛАБОРАТОРНАЯ РАБОТА № 3 · РАЗБОР КОДА

Программа nc_menu.py построчно

Меню без стандартных виджетов в стиле Norton Commander. Вариант 1 — переименование файла. В документе: как устроена программа, какие конструкции Python в ней используются, что происходит при нажатии клавиши и щелчке, и разбор каждой строки всех файлов папки lab3.

1. Файлы и как они связаны

Файл	Строк	Что в нём	Кто его использует
`nc_menu.py`	1024	сама программа: экран, меню, окна, события	запускается пользователем
`fileops.py`	287	логика без интерфейса: каталог, маски, проверка и переименование файлов	импортируется в `nc_menu.py` (копия из ЛР2 без изменений)
`make_demo.py`	94	создаёт тестовую папку `demo` с файлами	запускается перед работой
`test_fileops.py`	169	17 автоматических тестов `fileops.py`	`python -m unittest test_fileops`
`README.md`	—	краткое описание и клавиши	читается человеком

Разделение простое: `nc_menu.py` отвечает за то, **как выглядит и как управляется** программа, а `fileops.py` — за то, **что делается с файлами**. Окно никогда не переименовывает файлы само: оно собирает план «старое имя → новое имя» и передаёт его функциям `fileops`. Поэтому правила проверки имён и тексты ошибок у программ ЛР2 и ЛР3 одинаковые.

Из чего состоит nc_menu.py

Строки	Часть	Назначение
1–63	константы	шрифт, размеры, цвета, рамки, таблицы F-клавиш и действий, текст справки
66–153	`Screen`	текстовая сетка поверх холста: все примитивы рисования и области для мыши
155–221	`TextField`	своё поле ввода
224–280	`Dialog`	основа модальных окон
283–473	окна	`MessageDialog`, `RenameDialog`, `MaskDialog`, `GroupDialog`, `PathDialog`
476–611	`PullDownMenu`	строка меню и выпадающие меню
614–1002	`App`	состояние программы, действия, обработка событий, рисование панели
1005–1024	`main`	запуск

2. Как работает программа с событиями

Обычная консольная программа выполняется сверху вниз и заканчивается. Программа с окном устроена иначе: она создаёт окно, говорит библиотеке, какие функции вызывать при каких событиях, и уходит в **главный цикл** `root.mainloop()`. Дальше всем управляет Tkinter: ждёт, пока пользователь нажмёт клавишу или щёлкнет, и вызывает нашу функцию-обработчик. Обработчик меняет состояние (каталог, курсор, отметки, открытые окна) и перерисовывает экран.

Ключевая идея nc_menu.py: **экран не хранит элементы, он каждый раз рисуется заново**. После любого события метод `App.redraw` стирает холст и рисует всё по текущему состоянию: панель, строку сообщений, F-строку, меню, окна. Отдельных «объектов-кнопок» нет — есть данные и функция, которая их рисует.

Что даёт Tkinter и что делаем мы

Даёт Tkinter	Делаем сами
окно `tk.Tk` и холст `tk.Canvas`	строку меню, выпадающие меню, панель файлов, F-строку, окна, кнопки, поле ввода
`create_rectangle` и `create_text` — прямоугольник и надпись	всё перечисленное собрано только из них
события: клавиша, щелчок, двойной щелчок, правая кнопка, колесо, размер окна	решаем, что делать: куда ушла клавиша, по чему щёлкнули
`tk.PhotoImage`, `tkinter.font.Font`	значок окна рисуем по пикселям, по шрифту считаем размер клетки

Путь нажатия клавиши F2

1. Пользователь нажимает F2. Tkinter вызывает `App.on_key` (привязка в строке 638).
2. `on_key` проверяет по порядку: открыто ли окно (строка 839), открыто ли меню (841), F-клавиша ли это (843). F2 есть в словаре `FUNCTION_KEYS` → имя метода `rename_one`.
3. `getattr(self, 'rename_one')()` вызывает `App.rename_one` (строка 716).
4. `rename_one` берёт файл под курсором (`cursor_file`) и создаёт окно `RenameDialog`, `show_dialog` кладёт его в стопку `self.dialogs`.
5. `redraw` рисует экран и поверх — окно. Перед окном список областей мыши очищается: щёлкать можно только по окну.
6. Пользователь печатает новое имя. Каждая клавиша снова идёт в `on_key`, но теперь окно открыто → `Dialog.key` → `RenameDialog.key_body` → `TextField.key`: символ вставляется в поле.
7. Enter: `Dialog.key` нажимает кнопку с фокусом — `RenameDialog.accept` (строка 321). Он вызывает `fileops.check_plan`. Ошибка — текст в окне; успех — окно закрывается и вызывается `App.apply`.
8. `App.apply` вызывает `fileops.apply_plan` — файл переименован на диске; каталог перечитывается, в строке сообщений «Переименован 1 файл: … → …», экран перерисовывается.

Путь щелчка мыши

1. При каждой перерисовке всё, по чему можно щёлкнуть, регистрирует область: `screen.region(x, y, w, h, обработчик)` — строки файлов, заголовки столбцов, F-ячейки, названия меню, пункты, кнопки, поля.
2. Щелчок → `App.on_mouse` (строка 870) → `Screen.hit` (строка 144).
3. `hit` переводит пиксели в клетку и ищет область с конца списка — то, что нарисовано последним, лежит сверху.
4. Найденный обработчик вызывается; затем `redraw`. Например, щелчок по «F2Переим» вызывает тот же `rename_one`, что и клавиша.

Модальность окон

Окно «модальное», если пока оно открыто, остальная программа недоступна. В nc_menu.py это сделано двумя приёмами: клавиши уходят только верхнему окну (`on_key`, строка 839), а перед рисованием верхнего окна список областей мыши очищается (`redraw`, строка 903) и окно первым регистрирует область во весь экран с пустым обработчиком (строка 248) — щелчок мимо окна ничего не делает.

3. Словарик конструкций Python

Запись	Что значит	Пример из кода
`class Имя:` / `class Б(А):`	класс; класс-наследник получает всё от А	`class RenameDialog(Dialog)`
`def метод(self, ...)`	метод; `self` — сам объект	`def close(self)`
`__init__`	конструктор, вызывается при создании объекта	`Screen(canvas)`
`super().__init__(app)`	вызвать конструктор родителя	в каждом окне
`@dataclass(frozen=True)`	класс-запись: конструктор создаётся сам, поля не меняются	`Entry`, `Rename`
`@staticmethod`	метод без `self`	`draw_icon`
`lambda a, b: выражение`	короткая безымянная функция	`lambda cx, cy: self.click(cx)`
`lambda ..., index=index:`	запомнить текущее значение переменной цикла	области строк и кнопок
`[x for x in ...]`, `{...}`	генераторы списка, словаря, множества	`FUNCTION_KEYS`, `marked_files`
`text[a:b]`	срез строки или списка с a до b	вставка символа в поле
`A if условие else B`	условное выражение	цвет строки
`try: ... except Ошибка:`	попробовать; при ошибке выполнить другую ветку	`accept` в окнах
`raise FileError(...)`	создать ошибку	`fileops`
`with ... as ...:`	ресурс закроется автоматически	`os.scandir`
`set()`, `&=`, `^=`, `|`	множество; пересечение; добавить/убрать; объединение	отметки файлов
`getattr(объект, 'имя')`	атрибут или метод по имени-строке	F-клавиши
`enumerate`, `zip`, `reversed`	номер + элемент; два списка вместе; обратный порядок	циклы рисования
`'%d – %s' % (a, b)`	подстановка значений в строку	переключатели в окне группы
`x: int`, `-> str`	подсказки типов, на работу не влияют	везде
`if __name__ == '__main__':`	код выполняется только при прямом запуске файла	конец файлов

Понятия Tkinter

Понятие	Что это
`event.keysym`	имя клавиши: `F2`, `Return`, `BackSpace`, `Left`, `Prior` (PgUp), `Next` (PgDn)
`event.char`	набранный символ с учётом раскладки (буква, цифра) или пустая строка
`event.keycode`	код физической клавиши — одинаков в русской и английской раскладке
`event.state`	нажатые модификаторы; `state & 0x0004` — Ctrl, `state & 0x0001` — Shift
`bind('<KeyPress>', f)`	вызывать функцию f при событии
`return 'break'`	событие обработано, дальше не передавать
`mainloop()`	главный цикл: ждать событий и вызывать обработчики

4. Построчный разбор nc_menu.py

Код показан блоками с номерами строк, как в редакторе. Под каждым блоком — что делает каждая строка или группа строк. Пустые строки и комментарии тоже указаны, чтобы номера сходились без пропусков.

Строки 1–7. Строка документации модуля

```
1  """Управление файловым каталогом через меню в стиле Norton Commander — без стандартных виджетов.
2
3  Из библиотеки Tkinter берутся только окно и холст (Canvas) как поверхность рисования. Строка меню,
4  выпадающие меню, список файлов, строка функциональных клавиш, поля ввода и диалоговые окна
5  рисуются программой; нажатия клавиш и щелчки мыши она обрабатывает сама.
6  """
7
```

- 1–6: строка документации (docstring) — текст в тройных кавычках в самом начале файла. Python его не выполняет, это описание модуля: что делает программа и что из Tkinter берётся (только окно и холст).
- 7: пустая строка — отделяет описание от кода.

Строки 8–16. Подключение модулей

```
8  import ctypes
9  import os
10 import sys
11 import tkinter as tk
12 from tkinter import font as tkfont
13
14 import fileops
15 from fileops import FileError, Rename
16
```

- 8: `ctypes` — стандартный модуль для вызова функций Windows; нужен в `main()`, чтобы окно было чётким на экранах с масштабом 125–150 %.
- 9: `os` — работа с путями и файлами (имя каталога, родительский каталог, расширение).
- 10: `sys` — аргументы запуска (`sys.argv`) и код завершения (`sys.exit`).
- 11: `tkinter` под коротким именем `tk` — из него берём окно `tk.Tk`, холст `tk.Canvas` и картинку `tk.PhotoImage` для значка.
- 12: `tkinter.font` под именем `tkfont` — шрифт; по нему считаем размер одной клетки экрана.
- 13: пустая строка между стандартными модулями и нашими.
- 14: `fileops` — наш модуль из соседнего файла `fileops.py`: каталог, маски, проверка и переименование.
- 15: из него отдельно берём два класса, чтобы писать короче: `FileError` (ошибка с текстом для пользователя) и `Rename` (пара «старое имя → новое»).
- 16: пустая строка.

Строки 17–22. Общие настройки

```
17 TITLE = 'Управление файлами — меню без стандартных виджетов'
18 FONT = ('Consolas', 12)
19 START_COLS, START_ROWS = 100, 32
20 MIN_COLS, MIN_ROWS = 72, 22
21 PREVIEW_ROWS = 6
22
```

- 17: `TITLE` — заголовок окна. Имена из заглавных букв по соглашению Python означают константы — значения, которые не меняются во время работы.
- 18: `FONT` — шрифт Consolas размера 12. Он моноширинный: все символы одной ширины, поэтому экран можно считать сеткой клеток, как в текстовом режиме DOS.
- 19: размер окна при запуске — 100 столбцов на 32 строки (в клетках). Запись `a, b = 100, 32` присваивает два значения сразу.
- 20: минимальный размер окна — 72 × 22 клетки, меньше окно не сожмётся.
- 21: `PREVIEW_ROWS` — сколько строк предпросмотра показывать в окне переименования группы.
- 22: пустая строка.

Строки 23–30. Цвета и символы рамок

```
23  # цвета текстового режима Norton Commander
24  BLUE, CYAN, LIGHT_CYAN = '#0000a8', '#00a8a8', '#55ffff'
25  WHITE, BLACK, GRAY, DARK_GRAY = '#ffffff', '#000000', '#a8a8a8', '#545454'
26  YELLOW, RED, DARK_RED = '#ffff55', '#a80000', '#7a0000'
27
28  SINGLE = '┌─┐│└┘'
29  DOUBLE = '╔═╗║╚╝'
30
```

- 23: комментарий — строка после `#`, Python её пропускает.
- 24–26: цвета в формате `#RRGGBB` (красный, зелёный, синий в шестнадцатеричной записи). Это цвета текстового режима, как в Norton Commander: синий фон, бирюзовый, светло-бирюзовый, белый, чёрный, серый, тёмно-серый, жёлтый, красный, тёмно-красный.
- 27: пустая строка.
- 28: `SINGLE` — шесть символов одинарной рамки: левый верхний угол, горизонталь, правый верхний угол, вертикаль, левый нижний, правый нижний: `┌ ─ ┐ │ └ ┘`. Ими рисуется выпадающее меню.
- 29: `DOUBLE` — те же шесть символов двойной рамки: `╔ ═ ╗ ║ ╚ ╝` (в коде записаны подряд, без пробелов). Ими рисуются панель и окна.
- 30: пустая строка.

Строки 31–44. Таблицы настроек: сортировка, действия, F-клавиши

```
31  SORT_KEYS = [('name', 'по имени'), ('size', 'по размеру'), ('modified', 'по дате')]
32  ACTIONS = [
33      ('prefix', 'добавить текст в начало имени', 'Текст:'),
34      ('suffix', 'добавить текст в конец имени (перед расширением)', 'Текст:'),
35      ('extension', 'сменить расширение', 'Расширение:'),
36  ]
37  KEYBAR = [('1', 'Помощь', 'help'), ('2', 'Переим', 'rename_one'), ('3', 'Группа', 'rename_group'),
38            ('4', 'Маска', 'select_by_mask'), ('5', 'Обнов', 'refresh'), ('6', 'Путь', 'go_to_path'),
39            ('7', 'Сорт', 'next_sort'), ('8', '', None), ('9', 'Меню', 'open_menu'), ('10', 'Выход', 'quit')]
40  FUNCTION_KEYS = {'F%d' % (index + 1): action for index, (_, _, action) in enumerate(KEYBAR) if action}
41  # коды клавиш вместо букв, чтобы сочетания работали и в русской раскладке
42  CONTROL_A = 65
43  CONTROL_MASK, SHIFT_MASK = 0x0004, 0x0001
44
```

- 31: `SORT_KEYS` — способы сортировки: пары «ключ — подпись». Ключ совпадает с именем поля объекта каталога (`name`, `size`, `modified`).
- 32–36: `ACTIONS` — три действия окна переименования группы. Каждый элемент — кортеж из трёх частей: внутреннее имя, текст переключателя, подпись поля ввода.
- 37–39: `KEYBAR` — нижняя строка F-клавиш: номер, подпись и имя метода класса `App`, который вызывается. У восьмой клавиши действия нет (`None`) — ячейка пустая.
- 40: `FUNCTION_KEYS` — словарь, построенный из `KEYBAR` генератором словаря: «имя клавиши в Tkinter "F1" → имя метода». `enumerate` даёт номер элемента, `if action` отбрасывает пустую F8. По этому словарю `on_key` находит, что делать при нажатии F-клавиши.
- 41: комментарий: почему дальше используются коды клавиш.
- 42: `CONTROL_A = 65` — код клавиши A. В русской раскладке та же клавиша даёт букву «Ф», но код у неё тот же, поэтому Ctrl+A работает в любой раскладке.
- 43: битовые маски состояния модификаторов в событии Tkinter: 0x0004 — нажат Ctrl, 0x0001 — нажат Shift. Проверка делается побитовым «и»: `event.state & CONTROL_MASK`.
- 44: пустая строка.

Строки 45–65. Текст справки

```
45  HELP_LINES = [
46      'Программа показывает содержимое текущего каталога и переименовывает файлы.',
47      '',
48      'Перемещение по списку: ↑ ↓, PgUp, PgDn, Home, End, колесо мыши, щелчок.',
49      'Enter или двойной щелчок — войти в каталог; Backspace или строка «..» — вверх.',
50      'F6 — перейти в каталог по пути.',
51      '',
52      'F2 — переименовать файл под курсором (Enter на файле делает то же).',
53      'Insert, пробел или правая кнопка мыши — отметить файл. Ctrl+A — отметить все',
54      'файлы, F4 — отметить по маске, Esc — снять отметку.',
55      'F3 — переименовать отмеченные файлы: текст в начало, в конец или новое расширение.',
56      'Маска: * — любые символы, ? — один символ. Например: *.txt, IMG_*, отчёт_?.docx.',
57      '',
58      'Если хотя бы одно новое имя занято или недопустимо, не переименовывается ни один',
59      'файл. Каталоги программа не переименовывает.',
60      '',
61      'F9 — меню, F7 — сменить сортировку, F5 — обновить, F10 — выход.',
62      'В меню и окнах: стрелки, Enter, Esc и выделенные буквы.',
63  ]
64
65
```

- 45–63: `HELP_LINES` — список строк окна справки (F1). Пустые строки `''` дают пустые строки в окне.
- 64–65: две пустые строки — по правилам оформления Python перед определением класса.

Строки 66–76. Класс Screen: создание

```
66  class Screen:
67      """Текстовая сетка поверх холста: всё рисуется в клетках одинакового размера."""
68
69      def __init__(self, canvas: tk.Canvas):
70          self.canvas = canvas
71          self.font = tkfont.Font(family=FONT[0], size=FONT[1])
72          self.cell_w = self.font.measure('0')
73          self.cell_h = self.font.metrics('linespace')
74          self.cols, self.rows = START_COLS, START_ROWS
75          self.regions = []
76
```

- 66: `class Screen:` — начало класса. Screen — «текстовая сетка поверх холста»: через него рисуется всё.
- 67: строка документации класса.
- 68: пустая строка.
- 69: `__init__` — конструктор, вызывается при создании объекта `Screen(canvas)`. `self` — сам создаваемый объект; `canvas: tk.Canvas` — подсказка типа (на работу не влияет).
- 70: запоминаем холст, на котором будем рисовать.
- 71: создаём объект шрифта Consolas 12.
- 72: `measure('0')` — ширина символа в пикселях; это ширина клетки.
- 73: `metrics('linespace')` — высота строки шрифта; это высота клетки.
- 74: начальное число столбцов и строк сетки.
- 75: `regions` — список областей, по которым можно щёлкнуть мышью; заполняется при каждой перерисовке.
- 76: пустая строка.

Строки 77–85. Screen.resize и Screen.clear

```
77      def resize(self, width: int, height: int) -> None:
78          self.cols = max(width // self.cell_w, MIN_COLS)
79          self.rows = max(height // self.cell_h, MIN_ROWS)
80
81      def clear(self, color: str) -> None:
82          self.canvas.delete('all')
83          self.regions = []
84          self.canvas.configure(background=color)
85
```

- 77: `resize` вызывается при изменении размера окна; аргументы — новая ширина и высота в пикселях.
- 78: столбцов = ширина // ширина клетки (`//` — деление нацело), но не меньше минимума.
- 79: так же строки.
- 80: пустая строка.

- 81: `clear` — очистить экран перед новой перерисовкой.
- 82: `delete('all')` удаляет с холста все нарисованные фигуры и тексты.
- 83: список кликабельных областей тоже обнуляется — каждый кадр регистрирует их заново.
- 84: фон холста.
- 85: пустая строка.

Строки 86–98. Screen.fill и Screen.text — два кирпичика всей отрисовки

```
 86      def fill(self, x: int, y: int, w: int, h: int, color: str) -> None:
 87          if w > 0 and h > 0:
 88              self.canvas.create_rectangle(x * self.cell_w, y * self.cell_h, (x + w) * self.cell_w,
 89                                           (y + h) * self.cell_h, fill=color, width=0)
 90
 91      def text(self, x: int, y: int, text: str, fg: str, bg: str = None, width: int = None) -> None:
 92          if width is not None:
 93              text = (text[:width - 1] + '…') if len(text) > width else text.ljust(width)
 94          if bg is not None:
 95              self.fill(x, y, len(text), 1, bg)
 96          if text.strip():
 97              self.canvas.create_text(x * self.cell_w, y * self.cell_h, text=text, anchor='nw', fill=fg, font=self.font)
 98
```

- 86: `fill(x, y, w, h, color)` — закрасить прямоугольник из клеток: x, y — левый верхний угол в клетках, w, h — ширина и высота в клетках.
- 87: защита от пустого прямоугольника.
- 88–89: `create_rectangle` холста рисует прямоугольник; клетки переводятся в пиксели умножением на размер клетки. `width=0` — без обводки.
- 90: пустая строка.
- 91: `text(x, y, text, fg, bg, width)` — вывести строку в клетке (x, y) цветом `fg`; `bg` — цвет фона под текстом, `width` — ширина поля. `= None` — значения по умолчанию, эти аргументы можно не передавать.
- 92–93: если ширина задана: длинный текст обрезается и получает «…» в конце, короткий дополняется пробелами (`ljust`) — так строка закрашивается на всю ширину. Запись `A if условие else B` — условное выражение.
- 94–95: если задан фон — сначала закрашиваем под текстом прямоугольник.
- 96: пустую строку из пробелов не рисуем — экономия фигур на холсте.
- 97: `create_text` холста рисует надпись; `anchor='nw'` — координата задаёт левый верхний угол текста.
- 98: пустая строка.

Строки 99–107. Screen.label — текст с выделенной буквой

```
 99      def label(self, x: int, y: int, markup: str, fg: str, bg: str, accent: str) -> int:
100          """Текст, в котором буква после знака & выделена цветом. Возвращает ширину в клетках."""
101          plain = markup.replace('&', '')
102          self.text(x, y, plain, fg, bg)
103          if '&' in markup:
104              index = markup.index('&')
105              self.text(x + index, y, markup[index + 1], accent, bg)
106          return len(plain)
107
```

- 99: подпись вида `&Файл`: буква после знака & — «горячая», её можно нажать в открытом меню.
- 100: строка документации.
- 101: убираем & — получаем обычный текст.
- 102: рисуем его основным цветом.
- 103–105: если & был, поверх рисуем одну букву (стоящую за &) цветом выделения и тем же фоном — фон закрывает букву основного цвета.
- 106: возвращаем ширину подписи — она нужна, чтобы задать область для щелчка.
- 107: пустая строка.

Строки 108–120. Screen.frame — рамка

```
108     def frame(self, x: int, y: int, w: int, h: int, fg: str, bg: str, chars: str = DOUBLE, title: str = '',
109               title_fg: str = None) -> None:
110         top_left, line, top_right, side, bottom_left, bottom_right = chars
111         self.fill(x, y, w, h, bg)
112         self.text(x, y, top_left + line * (w - 2) + top_right, fg)
113         for row in range(y + 1, y + h - 1):
114             self.text(x, row, side, fg)
115             self.text(x + w - 1, row, side, fg)
116         self.text(x, y + h - 1, bottom_left + line * (w - 2) + bottom_right, fg)
117         if title:
118             title = ' ' + title[:w - 6] + ' '
119             self.text(x + (w - len(title)) // 2, y, title, title_fg or fg, bg)
120
```

- 108–109: рамка в прямоугольнике x, y, w, h; `chars` — набор символов (двойная по умолчанию), `title` — заголовок по центру верхней линии, `title_fg` — его цвет.
- 110: распаковка строки из шести символов в шесть переменных.
- 111: закрашиваем внутренность фоном.
- 112: верхняя линия: угол + горизонталь нужной длины (`строка * n` повторяет строку n раз) + угол.
- 113–115: для каждой строки между верхом и низом — вертикальные черты слева и справа.
- 116: нижняя линия.
- 117–119: заголовок: обрезается по ширине, обрамляется пробелами и ставится по центру верхней линии.
- 120: пустая строка.

Строки 121–135. Screen.wrapped — перенос по словам

```
121     def wrapped(self, x: int, y: int, text: str, fg: str, width: int, lines: int = 2) -> None:
122         """Текст с переносом по словам на несколько строк."""
123         rows, line = [], ''
124         for word in text.split():
125             if line and len(line) + 1 + len(word) > width:
126                 rows.append(line)
127                 line = word
128             else:
129                 line = (line + ' ' + word).strip()
130         rows.append(line)
131         if len(rows) > lines:
132             rows = rows[:lines - 1] + [' '.join(rows[lines - 1:])]
133         for row, part in enumerate(rows):
134             self.text(x, y + row, part, fg, width=width)
135
```

- 121–122: вывести длинное сообщение (например, об ошибке) на нескольких строках, не больше `lines`.
- 123: `rows` — готовые строки, `line` — строка, которая сейчас набирается.
- 124: перебираем слова сообщения.
- 125–127: если слово не помещается в текущую строку — строка готова, новое слово начинает следующую.
- 128–129: иначе добавляем слово через пробел; `strip()` убирает лишний пробел в начале.
- 130: последняя набранная строка тоже идёт в результат.
- 131–132: если строк больше разрешённого, хвост склеивается в последнюю строку (её обрежет `text`).
- 133–134: выводим строки одну под другой.
- 135: пустая строка.

Строки 136–153. Тень, области для мыши и поиск попадания

```
136     def shadow(self, x: int, y: int, w: int, h: int) -> None:
137         self.fill(x + w, y + 1, 2, h, '#202020')
138         self.fill(x + 2, y + h, w, 1, '#202020')
139 
140     def region(self, x: int, y: int, w: int, h: int, click, double=None, right=None) -> None:
141         """Область, которая реагирует на мышь. Проверяются в обратном порядке: верхний слой — первым."""
142         self.regions.append((x, y, x + w, y + h, click, double, right))
143 
144     def hit(self, px: int, py: int, kind: str) -> bool:
145         cx, cy = px // self.cell_w, py // self.cell_h
146         for x0, y0, x1, y1, click, double, right in reversed(self.regions):
147             if x0 <= cx < x1 and y0 <= cy < y1:
148                 handler = {'click': click, 'double': double or click, 'right': right}[kind]
149                 if handler is not None:
150                     handler(cx - x0, cy - y0)
151                 return True
152         return False
153 
```

- 136–138: `shadow` — тёмная полоса справа и снизу от окна, как тень в Norton Commander.
- 139: пустая строка.
- 140–142: `region` регистрирует прямоугольник, реагирующий на мышь, и до трёх обработчиков: обычный щелчок, двойной и правой кнопкой. Обработчик — функция, которую вызовут при щелчке.
- 143: пустая строка.
- 144: `hit` вызывается при щелчке: `px, py` — координаты в пикселях, `kind` — вид щелчка.
- 145: пиксели переводятся в номер клетки.
- 146: перебор областей с конца (`reversed`): то, что нарисовано позже, лежит сверху и должно получить щелчок первым.
- 147: попал ли щелчок в область.
- 148: выбираем обработчик по виду щелчка; для двойного щелчка, если отдельного нет, — обычный.
- 149–150: вызываем его, передав место щелчка внутри области.
- 151: область найдена — дальше не ищем (нижние слои щелчок не получат).
- 152: никуда не попали.
- 153: пустая строка.

Строки 154–172. Класс TextField: своё поле ввода

```
154 
155 class TextField:
156     """Однострочное поле ввода: текст, курсор и выделение хранятся и рисуются вручную."""
157 
158     def __init__(self, text: str = '', select=None):
159         self.text = text
160         self.pos = len(text) if select is None else select[1]
161         self.sel = select if select is not None else (0, len(text)) if text else None
162         self.offset = 0
163 
164     def _delete_selection(self) -> bool:
165         if not self.sel or self.sel[0] == self.sel[1]:
166             self.sel = None
167             return False
168         start, end = self.sel
169         self.text = self.text[:start] + self.text[end:]
170         self.pos, self.sel = start, None
171         return True
172 
```

- 154: пустая строка.
- 155–156: стандартного поля ввода (`Entry`) нет, поэтому поле устроено вручную: текст, позиция курсора и выделение — просто переменные, рисуются они тоже сами.
- 157: пустая строка.
- 158: конструктор: начальный текст и, если нужно, выделенный участок (`начало, конец`).
- 159: текст поля.
- 160: `pos` — позиция курсора (номер символа перед которым он стоит).

- 161: `sel` — выделение: заданное, иначе весь текст (если он есть), иначе ничего. Так при открытии окна набор сразу заменяет старое содержимое.
- 162: `offset` — с какого символа виден текст, если он длиннее поля (горизонтальная прокрутка).
- 163: пустая строка.
- 164–171: `_delete_selection` — удалить выделенный текст. Подчёркивание в начале имени — соглашение: метод внутренний. Если выделения нет или оно пустое — возвращает False; иначе вырезает участок срезами `text[:start] + text[end:]`, ставит курсор на место выделения и возвращает True.
- 172: пустая строка.

Строки 173–202. TextField.key — обработка клавиш поля

```
173     def key(self, event) -> bool:
174         """Обрабатывает клавишу; возвращает True, если она относилась к полю."""
175         keysym = event.keysym
176         if event.state & CONTROL_MASK and event.keycode == CONTROL_A:
177             self.sel, self.pos = (0, len(self.text)), len(self.text)
178         elif keysym == 'Left':
179             self.pos = self.sel[0] if self.sel else max(self.pos - 1, 0)
180             self.sel = None
181         elif keysym == 'Right':
182             self.pos = self.sel[1] if self.sel else min(self.pos + 1, len(self.text))
183             self.sel = None
184         elif keysym == 'Home':
185             self.pos, self.sel = 0, None
186         elif keysym == 'End':
187             self.pos, self.sel = len(self.text), None
188         elif keysym == 'BackSpace':
189             if not self._delete_selection() and self.pos > 0:
190                 self.text = self.text[:self.pos - 1] + self.text[self.pos:]
191                 self.pos -= 1
192         elif keysym == 'Delete':
193             if not self._delete_selection():
194                 self.text = self.text[:self.pos] + self.text[self.pos + 1:]
195         elif event.char and event.char.isprintable() and not event.state & CONTROL_MASK:
196             self._delete_selection()
197             self.text = self.text[:self.pos] + event.char + self.text[self.pos:]
198             self.pos += 1
199         else:
200             return False
201         return True
202
```

- 173–174: вызывается для каждой клавиши; возвращает True, если клавиша относилась к полю.
- 175: `keysym` — имя клавиши в Tkinter: `Left`, `BackSpace`, `F2`…
- 176–177: Ctrl+A — выделить весь текст.
- 178–180: ← — если есть выделение, курсор встаёт в его начало, иначе сдвигается влево (не меньше 0); выделение снимается.
- 181–183: → — аналогично вправо.
- 184–187: Home и End — в начало и в конец.
- 188–191: Backspace — удалить выделение, а если его нет — символ слева от курсора.
- 192–194: Delete — удалить выделение или символ справа от курсора.
- 195–198: обычный символ: `event.char` — набранный символ (с учётом раскладки, поэтому работает кириллица); `isprintable()` отсекает управляющие. Выделение заменяется, символ вставляется в позицию курсора, курсор сдвигается.
- 199–200: остальные клавиши (например, ↑ ↓) полю не нужны — False, их обработает окно.
- 201: клавиша обработана.
- 202: пустая строка.

Строки 203–223. TextField.draw и click — рисование поля

```
203     def draw(self, screen: Screen, x: int, y: int, width: int) -> None:
204         if self.pos < self.offset:
205             self.offset = self.pos
206         if self.pos >= self.offset + width:
207             self.offset = self.pos - width + 1
208         visible = self.text[self.offset:self.offset + width]
209         screen.text(x, y, visible, BLACK, CYAN, width)
210         if self.sel:
211             start = max(self.sel[0] - self.offset, 0)
212             end = min(self.sel[1] - self.offset, width)
213             if end > start:
214                 screen.text(x + start, y, visible[start:end], WHITE, BLUE)
215         cursor = self.pos - self.offset
216         char = self.text[self.pos] if self.pos < len(self.text) else ' '
217         screen.text(x + cursor, y, char, CYAN, BLACK)
218         screen.region(x, y, width, 1, lambda cx, cy: self.click(cx))
219 
220     def click(self, cx: int) -> None:
221         self.pos, self.sel = min(self.offset + cx, len(self.text)), None
222 
223 
```

- 203: нарисовать поле в клетке (x, y) шириной `width`.
- 204–207: прокрутка: если курсор ушёл левее видимой части или правее её — сдвигаем `offset`, чтобы курсор всегда был виден.
- 208: видимая часть текста.
- 209: фон поля — бирюзовый, текст чёрный.
- 210–214: выделенный участок перерисовывается белым на синем.
- 215–217: курсор — символ под ним, нарисованный «наоборот» (бирюзовый на чёрном), как блочный курсор в текстовом режиме. Если курсор в конце текста — рисуется пробел.
- 218: поле регистрирует область: щелчок ставит курсор. `lambda cx, cy: ...` — короткая безымянная функция.
- 219: пустая строка.
- 220–221: `click` — курсор в позицию щелчка (не дальше конца текста), выделение снимается.
- 222–223: пустые строки.

Строки 224–236. Класс Dialog — основа всех окон

```
224 class Dialog:
225     """Модальное окно, нарисованное на холсте. Пока оно открыто, клавиши и мышь получает только оно."""
226 
227     width, height = 60, 9
228     title = ''
229     colors = (BLACK, GRAY, WHITE)        # текст, фон, рамка
230 
231     def __init__(self, app):
232         self.app = app
233         self.buttons = []                # (подпись, действие)
234         self.focus = 0                   # номер кнопки, на которой фокус клавиатуры
235         self.message = ''
236 
```

- 224–225: модальное окно: пока оно открыто, всё вводимое получает только оно.
- 226: пустая строка.
- 227–229: атрибуты класса — значения по умолчанию для всех окон: размер, заголовок, цвета (текст, фон, рамка). Наследники переопределяют их.
- 230: пустая строка.
- 231–235: конструктор: ссылка на приложение `app` (через неё окно вызывает действия), список кнопок (пары «подпись — функция»), номер кнопки с фокусом и строка сообщения.
- 236: пустая строка.

Строки 237–256. Dialog: закрытие, положение и рисование

```
237     def close(self) -> None:
238         self.app.close_dialog(self)
239
240     def origin(self, screen: Screen):
241         width = min(self.width, screen.cols - 4)
242         height = min(self.height, screen.rows - 2)
243         return (screen.cols - width) // 2, (screen.rows - height) // 2, width, height
244
245     def draw(self, screen: Screen) -> None:
246         x, y, w, h = self.origin(screen)
247         fg, bg, border = self.colors
248         screen.region(0, 0, screen.cols, screen.rows, lambda cx, cy: None)   # мимо окна — ничего
249         screen.shadow(x, y, w, h)
250         screen.frame(x, y, w, h, border, bg, DOUBLE, self.title, fg)
251         self.draw_body(screen, x + 2, y + 1, w - 4)
252         self.draw_buttons(screen, x, y + h - 2, w)
253
254     def draw_body(self, screen: Screen, x: int, y: int, width: int) -> None:
255         pass
256
```

- 237–238: `close` — попросить приложение убрать окно из стопки открытых.
- 239: пустая строка.
- 240–243: `origin` — где разместить окно: размер не больше экрана, положение — по центру. Возвращает четыре числа.
- 244: пустая строка.
- 245: `draw` — рисует окно целиком.
- 246–247: распаковка положения и цветов.
- 248: первой регистрируется область во весь экран с пустым обработчиком: щелчок мимо окна «съедается» и ни на что не действует — так окно становится модальным для мыши.
- 249–250: тень и рамка с заголовком.
- 251: содержимое рисует наследник в методе `draw_body` (отступы: 2 клетки слева, 1 сверху).
- 252: кнопки — в предпоследней строке окна.
- 253: пустая строка.
- 254–255: `draw_body` здесь пустой (`pass` — «ничего не делать»); каждое окно определяет свой.
- 256: пустая строка.

Строки 257–282. Dialog: кнопки и клавиши

```
257     def draw_buttons(self, screen: Screen, x: int, y: int, w: int) -> None:
258         labels = ['[ %s ]' % text for text, _ in self.buttons]
259         total = sum(len(label) for label in labels) + 2 * (len(labels) - 1)
260         cx = x + (w - total) // 2
261         fg, bg, _ = self.colors
262         for index, (label, (_, action)) in enumerate(zip(labels, self.buttons)):
263             focused = index == self.focus
264             screen.text(cx, y, label, BLACK if focused else fg, CYAN if focused else bg)
265             screen.region(cx, y, len(label), 1, lambda _x, _y, action=action: action())
266             cx += len(label) + 2
267
268     def key(self, event) -> None:
269         if event.keysym == 'Escape':
270             self.close()
271         elif event.keysym == 'Tab' and self.buttons:
272             step = -1 if event.state & SHIFT_MASK else 1
273             self.focus = (self.focus + step) % len(self.buttons)
274         elif event.keysym in ('Return', 'KP_Enter') and self.buttons:
275             self.buttons[self.focus][1]()
276         else:
277             self.key_body(event)
278
279     def key_body(self, event) -> None:
280         pass
281
282
```

- 257: `draw_buttons` — нарисовать кнопки по центру строки.
- 258: подписи вида «[OK]».
- 259: общая ширина кнопок с промежутками по 2 клетки.

- 260: левый край первой кнопки.
- 261: цвета окна.
- 262: перебор кнопок; `zip` идёт по двум спискам параллельно.
- 263–264: кнопка с фокусом рисуется чёрным на бирюзовом, остальные — цветами окна.
- 265: область кнопки: щелчок вызывает её действие. `action=action` в lambda — приём, который запоминает текущее значение переменной цикла (иначе все кнопки вызвали бы последнее действие).
- 266: сдвиг к следующей кнопке.
- 267: пустая строка.
- 268–270: `key` — клавиши окна: Esc закрывает.
- 271–273: Tab — фокус на следующую кнопку, Shift+Tab — на предыдущую (`%` — остаток от деления, чтобы зациклить).
- 274–275: Enter — нажать кнопку с фокусом.
- 276–277: остальные клавиши — в `key_body` конкретного окна (например, в поле ввода).
- 278: пустая строка.
- 279–280: `key_body` по умолчанию ничего не делает.
- 281–282: пустые строки.

Строки 283–299. MessageDialog — справка, ошибка, «О программе»

```
283  class MessageDialog(Dialog):
284      def __init__(self, app, title: str, lines: list, error: bool = False):
285          super().__init__(app)
286          self.title = title
287          self.lines = lines
288          self.width = min(max(max(len(line) for line in lines) + 6, 40), 92)
289          self.height = len(lines) + 5
290          if error:
291              self.colors = (WHITE, RED, WHITE)
292          self.buttons = [('OK', self.close)]
293
294      def draw_body(self, screen, x, y, width):
295          fg = self.colors[0]
296          for row, line in enumerate(self.lines):
297              screen.text(x, y + 1 + row, line, fg, width=width)
298
299
```

- 283: `class MessageDialog(Dialog)` — наследник: получает всё от Dialog и добавляет своё.
- 284–285: конструктор; `super().__init__(app)` вызывает конструктор Dialog.
- 286–287: заголовок и строки текста.
- 288: ширина по самой длинной строке плюс поля, но от 40 до 92 клеток.
- 289: высота — число строк плюс рамка и кнопка.
- 290–291: окно ошибки — белое на красном.
- 292: одна кнопка OK, закрывающая окно.
- 293: пустая строка.
- 294–297: `draw_body` выводит строки текста одна под другой.
- 298–299: пустые строки.

Строки 300–330. RenameDialog — переименование одного файла (F2)

```
300  class RenameDialog(Dialog):
301      title = 'Переименование файла'
302      width, height = 66, 11
303
304      def __init__(self, app, entry: fileops.Entry):
305          super().__init__(app)
306          self.entry = entry
307          base = len(os.path.splitext(entry.name)[0])
308          self.field = TextField(entry.name, (0, base))
309          self.buttons = [('Переименовать', self.accept), ('Отмена', self.close)]
310
311      def draw_body(self, screen, x, y, width):
312          screen.text(x, y + 1, 'Файл: ' + self.entry.name, BLACK, width=width)
313          screen.text(x, y + 3, 'Новое имя:', BLACK)
314          self.field.draw(screen, x + 11, y + 3, width - 11)
315          screen.wrapped(x, y + 5, self.message, DARK_RED, width)
316
317      def key_body(self, event):
318          if self.field.key(event):
319              self.message = ''
320
321      def accept(self):
322          try:
323              changes = fileops.check_plan(self.app.entries, [Rename(self.entry.name, self.field.text.strip())])
324          except FileError as error:
325              self.message = str(error)
326              return
327          self.close()
328          self.app.apply(changes)
329
330
```

- 300–302: заголовок и размер окна.
- 303: пустая строка.
- 304: конструктор получает файл (`Entry`) под курсором.
- 305–306: базовая часть и запоминание файла.
- 307: длина имени без расширения: `os.path.splitext('заметки.txt')` → `('заметки', '.txt')`.
- 308: поле с текущим именем, выделена часть до расширения — новое имя можно сразу печатать.
- 309: кнопки «Переименовать» и «Отмена».
- 310: пустая строка.
- 311–315: содержимое: строка «Файл: …», подпись и поле «Новое имя:», ниже — сообщение об ошибке красным с переносом.
- 316: пустая строка.
- 317–319: клавиши идут в поле; если поле их приняло, старое сообщение об ошибке стирается.
- 320: пустая строка.
- 321: `accept` — нажата «Переименовать» или Enter.
- 322–323: `try` — попытка. План из одного шага «старое → новое» проверяется функцией `check_plan` из `fileops`: занятое имя, запрещённые символы и т. п.
- 324–326: `except FileError` — если проверка не прошла, текст ошибки показывается в окне, окно остаётся.
- 327–328: проверка прошла — окно закрывается, приложение выполняет переименование.
- 329–330: пустые строки.

Строки 331–371. MaskDialog — отметить файлы по маске (F4)

```
331  class MaskDialog(Dialog):
332      title = 'Отметить файлы по маске'
333      width, height = 62, 11
334
335      def __init__(self, app):
336          super().__init__(app)
337          self.field = TextField('*')
338          self.buttons = [('Отметить', self.accept), ('Отмена', self.close)]
339
340      def matching(self):
341          mask = self.field.text.strip()
342          if not mask:
343              raise FileError('Введите маску.')
344          found = fileops.select(self.app.entries, mask, files_only=True)
345          if not found:
346              raise FileError('Нет файлов, соответствующих маске: ' + mask)
347          return found
348
349      def draw_body(self, screen, x, y, width):
350          screen.text(x, y + 1, 'Маска:', BLACK)
351          self.field.draw(screen, x + 8, y + 1, width - 8)
352          screen.text(x + 8, y + 2, '* — любые символы, ? — один символ', DARK_GRAY)
353          screen.text(x + 8, y + 3, 'Например: *.txt, IMG_*, отчёт_?.docx', DARK_GRAY)
354          try:
355              summary, color = 'Подходит: ' + fileops.files_text(len(self.matching())), BLACK
356          except FileError as error:
357              summary, color = str(error), DARK_RED
358          screen.wrapped(x, y + 5, summary, color, width)
359
360      def key_body(self, event):
361          self.field.key(event)
362
363      def accept(self):
364          try:
365              found = self.matching()
366          except FileError:
367              return
368          self.close()
369          self.app.mark_entries(found)
370
371
```

- 331–333: заголовок и размер.
- 334: пустая строка.
- 335–338: поле с маской * (выделена — набор заменит её) и кнопки.
- 339: пустая строка.
- 340: `matching` — какие файлы подходят под маску.
- 341: маска без пробелов по краям.
- 342–343: пустая маска — ошибка. `raise` создаёт ошибку, её поймает вызывающий код.
- 344: `fileops.select` отбирает объекты, подходящие под маску; `files_only=True` — только файлы.
- 345–346: ничего не подошло — ошибка с текстом из согласованной таблицы ошибок варианта.
- 347: список подошедших файлов.
- 348: пустая строка.
- 349–353: подпись и поле маски, ниже серые подсказки.
- 354–358: при каждой перерисовке пробуем найти файлы: успех — «Подходит: N файлов» чёрным, ошибка — её текст красным. Так счётчик обновляется после каждой нажатой клавиши.
- 359: пустая строка.
- 360–361: клавиши — в поле.
- 362: пустая строка.
- 363–369: «Отметить»: если файлы есть — окно закрывается, приложение отмечает их; если нет — ничего не происходит, окно остаётся с сообщением.
- 370–371: пустые строки.

Строки 372–401. GroupDialog — группа файлов (F3): создание и план

```
372  class GroupDialog(Dialog):
373      title = 'Переименование группы файлов'
374
375      def __init__(self, app, files: list):
376          super().__init__(app)
377          self.files = files
378          self.action = 0
379          self.field = TextField()
380          self.width = 78
381          self.height = 15 + min(len(files), PREVIEW_ROWS)
382          self.buttons = [('Переименовать', self.accept), ('Отмена', self.close)]
383
384      def new_name(self, name: str) -> str:
385          key, text = ACTIONS[self.action][0], self.field.text
386          if key == 'prefix':
387              return fileops.add_prefix(name, text)
388          if key == 'suffix':
389              return fileops.add_suffix(name, text)
390          return fileops.change_extension(name, text)
391
392      def plan(self):
393          """Проверенный план или текст, почему переименовать нельзя."""
394          plan = [Rename(file.name, self.new_name(file.name)) for file in self.files]
395          if not self.field.text:
396              return plan, None, 'Заполните поле «' + ACTIONS[self.action][2].rstrip(':') + '».'
397          try:
398              return plan, fileops.check_plan(self.app.entries, plan), None
399          except FileError as error:
400              return plan, None, str(error)
401
```

- 372–373: окно переименования отмеченных файлов.
- 374: пустая строка.
- 375–382: конструктор: список файлов, номер выбранного действия (0 — первое), пустое поле, ширина 78, высота зависит от числа строк предпросмотра, кнопки.
- 383: пустая строка.
- 384–390: `new_name` — новое имя одного файла для выбранного действия. Само преобразование делают функции `fileops`: `add_prefix` (в начало), `add_suffix` (в конец перед расширением), `change_extension`.
- 391: пустая строка.
- 392–393: `plan` — составить план и проверить его.
- 394: генератор списка: для каждого файла пара «старое → новое».
- 395–396: поле пустое — план не проверяем, возвращаем подсказку «Заполните поле…».
- 397–400: иначе `check_plan`; результат — тройка: план, проверенные изменения (или None) и текст проблемы (или None).
- 401: пустая строка.

Строки 402–447. GroupDialog: рисование, выбор действия, клавиши

```
402     def draw_body(self, screen, x, y, width):
403         screen.text(x, y, 'Отмечено: ' + fileops.files_text(len(self.files)), BLACK)
404         for index, (_, title, _) in enumerate(ACTIONS):
405             mark = '(•)' if index == self.action else '( )'
406             line = '%s %d – %s' % (mark, index + 1, title)
407             screen.text(x, y + 2 + index, line, BLACK, CYAN if index == self.action else None, width=width)
408             screen.region(x, y + 2 + index, width, 1, lambda _x, _y, index=index: self.choose(index))
409         label = ACTIONS[self.action][2]
410         screen.text(x, y + 6, label, BLACK)
411         self.field.draw(screen, x + 13, y + 6, width - 13)
412 
413         plan, changes, problem = self.plan()
414         half = (width - 3) // 2
415         screen.text(x, y + 8, 'Старое имя'.ljust(half) + ' → ' + 'Новое имя', DARK_GRAY, width=width)
416         for row, step in enumerate(plan[:PREVIEW_ROWS]):
417             old = step.old if len(step.old) <= half else step.old[:half - 1] + '…'
418             screen.text(x, y + 9 + row, old.ljust(half) + ' → ' + step.new, BLACK, width=width)
419         rows = min(len(plan), PREVIEW_ROWS)
420         if len(plan) > PREVIEW_ROWS:
421             screen.text(x + half + 3, y + 8, '(и ещё %d)' % (len(plan) - PREVIEW_ROWS), DARK_GRAY)
422         if problem:
423             screen.wrapped(x, y + 10 + rows, problem, DARK_RED if self.field.text else DARK_GRAY, width)
424         else:
425             screen.text(x, y + 10 + rows, 'Будет переименовано: ' + fileops.files_text(len(changes)), BLACK,
426                         width=width)
427 
428     def choose(self, index: int) -> None:
429         self.action = index
430 
431     def key_body(self, event):
432         if event.keysym == 'Up':
433             self.action = (self.action - 1) % len(ACTIONS)
434         elif event.keysym == 'Down':
435             self.action = (self.action + 1) % len(ACTIONS)
436         elif not self.field.text and event.char in '123' and event.char:
437             self.action = int(event.char) - 1        # пока поле пустое, цифра выбирает действие
438         else:
439             self.field.key(event)
440 
441     def accept(self):
442         _, changes, _ = self.plan()
443         if changes:
444             self.close()
445             self.app.apply(changes)
446 
447 
```

- 402–403: строка «Отмечено: N файлов».
- 404–408: три переключателя: «(•)» у выбранного, «( )» у остальных; выбранный подсвечен; у каждого своя область — по щелчку выбирается действие.
- 409–411: подпись и поле ввода под выбранное действие («Текст:» или «Расширение:»).
- 412: пустая строка.
- 413: план считается заново при каждой перерисовке — предпросмотр всегда соответствует введённому тексту.
- 414: ширина колонки «Старое имя».
- 415: заголовок предпросмотра.
- 416–418: до 6 строк «старое → новое», длинное старое имя обрезается.
- 419–421: если файлов больше 6 — пометка «(и ещё N)».
- 422–426: под таблицей — проблема (красным; пока поле пустое — серой подсказкой) или «Будет переименовано: N файлов».
- 427: пустая строка.
- 428–429: `choose` — выбрать действие щелчком.
- 430: пустая строка.
- 431–435: ↑ и ↓ переключают действие по кругу.
- 436–437: пока поле пустое, цифра 1–3 выбирает действие (так считалось в KLM: F3, затем «3»).
- 438–439: остальное — в поле ввода.
- 440: пустая строка.

- 441–445: «Переименовать»: только если план прошёл проверку — окно закрывается, приложение переименовывает.
- 446–447: пустые строки.

Строки 448–475. PathDialog — перейти в каталог по пути (F6)

```
448  class PathDialog(Dialog):
449      title = 'Перейти в каталог'
450      width, height = 76, 10
451
452      def __init__(self, app):
453          super().__init__(app)
454          self.field = TextField(app.directory)
455          self.buttons = [('Перейти', self.accept), ('Отмена', self.close)]
456
457      def draw_body(self, screen, x, y, width):
458          screen.text(x, y + 1, 'Путь: относительный (Фото, ..) или абсолютный (C:\\Windows)', BLACK, width=width)
459          self.field.draw(screen, x, y + 2, width)
460          screen.wrapped(x, y + 4, self.message, DARK_RED, width)
461
462      def key_body(self, event):
463          if self.field.key(event):
464              self.message = ''
465
466      def accept(self):
467          try:
468              directory = fileops.resolve_directory(self.app.directory, self.field.text.strip())
469          except FileError as error:
470              self.message = str(error)
471              return
472          self.close()
473          self.app.open_directory(directory)
474
475
```

- 448–450: заголовок и размер.
- 451: пустая строка.
- 452–455: поле с текущим путём (выделено целиком) и кнопки.
- 456: пустая строка.
- 457–460: подсказка про относительный и абсолютный путь, поле, сообщение об ошибке. `\\` в строке Python — это один обратный слеш.
- 461: пустая строка.
- 462–464: клавиши — в поле, ошибка стирается.
- 465: пустая строка.
- 466–473: `fileops.resolve_directory` превращает введённый путь в настоящий каталог; ошибка («Каталог не найден») — в окне, иначе окно закрывается и каталог открывается.
- 474–475: пустые строки.

Строки 476–516. PullDownMenu: строка меню и список пунктов

```
476  class PullDownMenu:
477      """Строка меню и выпадающие меню. Пункт: (подпись с &, клавиша, действие, доступен ли, отмечен ли)."""
478
479      def __init__(self, app):
480          self.app = app
481          self.opened = None               # номер открытого меню или None
482          self.item = 0
483
484      def menus(self):
485          app = self.app
486          sort = app.sort_key
487          return [
488              ('&Файл', [
489                  ('&Перейти в каталог…', 'F6', app.go_to_path, True, False),
490                  ('&Вверх', 'Backspace', app.go_up, True, False),
491                  ('&Обновить', 'F5', app.refresh, True, False),
492                  None,
493                  ('В&ыход', 'F10', app.quit, True, False),
494              ]),
495              ('&Правка', [
496                  ('&Переименовать…', 'F2', app.rename_one, app.cursor_file() is not None, False),
497                  ('Переименовать &группу…', 'F3', app.rename_group, bool(app.marked), False),
498                  None,
499                  ('&Отметить / снять', 'Ins', app.toggle_mark, app.cursor_file() is not None, False),
500                  ('Отметить &все', 'Ctrl+A', app.mark_all, True, False),
501                  ('Отметить по &маске…', 'F4', app.select_by_mask, True, False),
502                  ('&Снять отметку', 'Esc', app.clear_marks, bool(app.marked), False),
503              ]),
504              ('&Вид', [
505                  ('По &имени', '', lambda: app.set_sort('name'), True, sort == 'name'),
506                  ('По &размеру', '', lambda: app.set_sort('size'), True, sort == 'size'),
507                  ('По &дате', '', lambda: app.set_sort('modified'), True, sort == 'modified'),
508                  None,
509                  ('По &убыванию', '', app.toggle_descending, True, app.descending),
510              ]),
511              ('&Справка', [
512                  ('&Как пользоваться', 'F1', app.help, True, False),
513                  ('&О программе', '', app.about, True, False),
514              ]),
515          ]
516
```

- 476–477: класс меню. Пункт меню — кортеж из пяти частей: подпись с &, подпись клавиши, действие, доступен ли, отмечен ли (√).
- 478: пустая строка.
- 479–482: `opened` — номер открытого меню или `None` (закрыто), `item` — выбранный пункт.
- 483: пустая строка.
- 484: `menus` строит описание всех меню заново при каждом вызове — поэтому доступность пунктов всегда актуальна.
- 485–486: короткие имена для приложения и текущей сортировки.
- 487–494: меню «Файл»: перейти, вверх, обновить, разделитель (`None`), выход.
- 495–503: меню «Правка». «Переименовать…» доступен, только если курсор на файле; «группу…» и «Снять отметку» — только если есть отмеченные файлы (`bool(app.marked)`).
- 504–510: меню «Вид»: сортировки отмечаются «√» по совпадению с текущей. `lambda: app.set_sort('name')` — функция без аргументов, которая при вызове задаст сортировку.
- 511–514: меню «Справка».
- 515–516: конец списка и пустая строка.

Строки 517–566. PullDownMenu: открытие, закрытие и рисование

```
517     def positions(self):
518         x, result = 2, []
519         for title, _ in self.menus():
520             result.append(x)
521             x += len(title.replace('&', '')) + 4
522         return result
523 
524     def open(self, index: int = 0) -> None:
525         self.opened = index
526         items = self.menus()[index][1]
527         self.item = next(n for n, item in enumerate(items) if item and item[3])
528 
529     def close(self) -> None:
530         self.opened = None
531 
532     def draw_bar(self, screen: Screen) -> None:
533         screen.text(0, 0, '', BLACK, CYAN, screen.cols)
534         for index, ((title, _), x) in enumerate(zip(self.menus(), self.positions())):
535             active = index == self.opened
536             bg = BLACK if active else CYAN
537             width = screen.label(x - 1, 0, ' ' + title + ' ', WHITE if active else BLACK, bg, YELLOW if active else WHITE)
538             screen.region(x - 1, 0, width, 1, lambda _x, _y, index=index: self.toggle(index))
539 
540     def draw_drop(self, screen: Screen) -> None:
541         if self.opened is None:
542             return
543         items = self.menus()[self.opened][1]
544         x = self.positions()[self.opened] - 1
545         width = max(len(item[0].replace('&', '')) + len(item[1]) + 8 for item in items if item)
546         height = len(items) + 2
547         screen.region(0, 0, screen.cols, screen.rows, lambda cx, cy: self.close())   # щелчок мимо закрывает
548         self.draw_bar(screen)                       # строку меню можно щёлкать и при открытом меню
549         screen.shadow(x, 1, width, height)
550         screen.frame(x, 1, width, height, BLACK, CYAN, SINGLE)
551         for row, item in enumerate(items):
552             y = 2 + row
553             if item is None:
554                 screen.text(x, y, '├' + '─' * (width - 2) + '┤', BLACK)
555                 continue
556             title, hotkey, action, enabled, checked = item
557             selected = row == self.item
558             fg = (WHITE if selected else BLACK) if enabled else DARK_GRAY
559             bg = BLACK if selected else CYAN
560             screen.text(x + 1, y, '', fg, bg, width - 2)
561             screen.text(x + 1, y, '√' if checked else ' ', fg)
562             screen.label(x + 3, y, title, fg, bg, (YELLOW if selected else WHITE) if enabled else DARK_GRAY)
563             screen.text(x + width - 2 - len(hotkey), y, hotkey, fg)
564             if enabled:
565                 screen.region(x + 1, y, width - 2, 1, lambda _x, _y, row=row: self.run(row))
566 
```

- 517–522: `positions` — столбец, с которого начинается название каждого меню: названия идут подряд с промежутком.
- 523: пустая строка.
- 524–527: `open` — открыть меню с номером `index` и выбрать первый доступный пункт (`next` берёт первый подходящий элемент генератора).
- 528: пустая строка.
- 529–530: `close` — закрыть.
- 531: пустая строка.
- 532–538: строка меню: бирюзовая полоса на весь экран; названия; открытое — белое на чёрном; у каждого названия область — щелчок открывает или закрывает это меню.
- 539: пустая строка.
- 540–542: выпадающий список рисуется, только если меню открыто.
- 543–546: пункты открытого меню, левый край, ширина по самому длинному пункту, высота.
- 547: область во весь экран: щелчок мимо меню закрывает его.
- 548: строка меню рисуется ещё раз поверх этой области — чтобы по названиям можно было щёлкать при открытом меню.
- 549–550: тень и одинарная рамка.
- 551–555: перебор пунктов; разделитель — линия « ├────┤ ».

- 556–563: обычный пункт: распаковка; выбранный — белым на чёрном, недоступный — серым; отметка «√»; подпись с выделенной буквой; подпись клавиши справа.
- 564–565: только доступный пункт получает область для щелчка.
- 566: пустая строка.

Строки 567–613. PullDownMenu: действия и клавиши

```
567     def toggle(self, index: int) -> None:
568         if self.opened == index:
569             self.close()
570         else:
571             self.open(index)
572 
573     def run(self, row: int) -> None:
574         action = self.menus()[self.opened][1][row][2]
575         self.close()
576         action()
577 
578     def move(self, step: int) -> None:
579         items = self.menus()[self.opened][1]
580         row = self.item
581         for _ in items:
582             row = (row + step) % len(items)
583             if items[row] and items[row][3]:
584                 self.item = row
585                 return
586 
587     def key(self, event) -> None:
588         keysym = event.keysym
589         menus = self.menus()
590         if keysym in ('Escape', 'F9', 'F10'):
591             self.close()
592         elif keysym == 'Left':
593             self.open((self.opened - 1) % len(menus))
594         elif keysym == 'Right':
595             self.open((self.opened + 1) % len(menus))
596         elif keysym == 'Up':
597             self.move(-1)
598         elif keysym == 'Down':
599             self.move(1)
600         elif keysym in ('Return', 'KP_Enter'):
601             self.run(self.item)
602         elif event.char:
603             letter = event.char.lower()
604             for row, item in enumerate(menus[self.opened][1]):
605                 if item and item[3] and '&' in item[0] and item[0][item[0].index('&') + 1].lower() == letter:
606                     self.run(row)
607                     return
608             for index, (title, _) in enumerate(menus):
609                 if title[title.index('&') + 1].lower() == letter:
610                     self.open(index)
611                     return
612 
613 
```

- 567–571: `toggle` — щелчок по названию: открыто — закрыть, иначе открыть.
- 572: пустая строка.
- 573–576: `run` — выполнить пункт: меню закрывается, затем вызывается действие.
- 577: пустая строка.
- 578–585: `move` — перейти к соседнему пункту (вверх или вниз по кругу), пропуская разделители и недоступные.
- 586: пустая строка.
- 587–589: `key` — клавиши при открытом меню.
- 590–591: Esc, F9 или F10 закрывают меню.
- 592–595: ← и → — соседнее меню.
- 596–599: ↑ и ↓ — соседний пункт.
- 600–601: Enter — выполнить выбранный пункт.
- 602–607: буква — выполнить доступный пункт, у которого эта буква после &. `lower()` — сравнение без учёта регистра.
- 608–611: или открыть меню, у названия которого эта буква.

- 612–613: пустые строки.

Строки 614–646. Класс App: создание окна и привязка событий

```
614  class App:
615      def __init__(self, root: tk.Tk, directory: str):
616          self.root = root
617          self.canvas = tk.Canvas(root, highlightthickness=0, borderwidth=0, background=BLUE)
618          self.canvas.pack(fill='both', expand=True)
619          self.screen = Screen(self.canvas)
620          self.menu = PullDownMenu(self)
621          self.dialogs = []
622          self.directory = directory
623          self.entries = []
624          self.items = []                 # строки панели: None — «..», дальше объекты каталога
625          self.cursor = 0
626          self.top = 0
627          self.marked = set()
628          self.sort_key = 'name'
629          self.descending = False
630          self.message = ''
631          self.alive = True
632
633          root.title(TITLE)
634          root.geometry('%dx%d' % (START_COLS * self.screen.cell_w, START_ROWS * self.screen.cell_h))
635          root.minsize(MIN_COLS * self.screen.cell_w, MIN_ROWS * self.screen.cell_h)
636          self.icon = self.draw_icon()
637          root.iconphoto(True, self.icon)
638          self.canvas.bind('<KeyPress>', self.on_key)
639          self.canvas.bind('<Button-1>', lambda event: self.on_mouse(event, 'click'))
640          self.canvas.bind('<Double-Button-1>', lambda event: self.on_mouse(event, 'double'))
641          self.canvas.bind('<Button-3>', lambda event: self.on_mouse(event, 'right'))
642          self.canvas.bind('<MouseWheel>', self.on_wheel)
643          self.canvas.bind('<Configure>', self.on_resize)
644          self.canvas.focus_set()
645          self.open_directory(directory)
646
```

- 614: `App` — главный класс программы: хранит состояние и связывает всё вместе.
- 615: конструктор получает окно Tk и начальный каталог.
- 616: запоминаем окно.
- 617: **единственный виджет внутри окна** — холст. Без рамки и подсветки, фон синий.
- 618: `pack(fill='both', expand=True)` — холст занимает всё окно и растягивается вместе с ним.
- 619: сетка поверх холста.
- 620: меню.
- 621: стопка открытых окон (последнее — верхнее).
- 622–624: текущий каталог, его объекты и строки панели (`None` в начале — строка «..»).
- 625–626: `cursor` — номер строки под курсором, `top` — номер первой видимой строки (прокрутка).
- 627: `marked` — множество имён отмеченных файлов (`set` — без повторов, быстрая проверка «есть ли»).
- 628–629: сортировка: по какому полю и по убыванию ли.
- 630: сообщение в строке состояния.
- 631: `alive` — окно ещё существует (после выхода рисовать нельзя).
- 632: пустая строка.
- 633–635: заголовок, начальный и минимальный размер окна в пикселях (клетки × размер клетки).
- 636–637: значок окна, нарисованный в коде (`draw_icon`).
- 638: **одна привязка клавиатуры**: любое нажатие на холсте вызывает `on_key`.
- 639–641: щелчок левой кнопкой, двойной щелчок и правая кнопка — все через `on_mouse` с видом щелчка.
- 642: колесо мыши.
- 643: `<Configure>` — изменение размера окна.
- 644: фокус клавиатуры на холсте — иначе нажатия до него не дойдут.
- 645: открыть начальный каталог.
- 646: пустая строка.

Строки 647–658. App.draw_icon — значок окна

```
647     @staticmethod
648     def draw_icon() -> tk.PhotoImage:
649         image = tk.PhotoImage(width=32, height=32)
650         image.put(BLUE, to=(2, 4, 30, 28))
651         image.put(LIGHT_CYAN, to=(4, 6, 28, 8))
652         for row in range(11, 26, 4):
653             image.put(LIGHT_CYAN, to=(6, row, 24, row + 2))
654         image.put(YELLOW, to=(6, 15, 24, 17))
655         return image
656
657     # --- состояние ---
658
```

- 647: `@staticmethod` — метод, которому не нужен `self`.
- 648–649: картинка 32 × 32 пикселя.
- 650–654: `put(цвет, to=(x1, y1, x2, y2))` закрашивает прямоугольник пикселей: синяя панель, полоса сверху, строки списка, жёлтая «отмеченная» строка — маленькая копия окна.
- 655: вернуть картинку.
- 656: пустая строка.
- 657: комментарий-раздел.
- 658: пустая строка.

Строки 659–692. App.open_directory и fill_items — открыть каталог и заполнить список

```
659     def open_directory(self, directory: str, select=()) -> bool:
660         try:
661             entries = fileops.list_directory(directory)
662         except FileError as error:
663             self.show_error(str(error))
664             return False
665         if directory != self.directory:
666             self.marked = set()
667             self.message = ''
668             self.items = []
669         self.directory = directory
670         self.entries = entries
671         self.marked &= {entry.name for entry in entries if not entry.is_dir}
672         self.root.title(TITLE + ' — ' + directory)
673         self.fill_items(select)
674         return True
675
676     def fill_items(self, select=()) -> None:
677         current = self.cursor_entry()
678         key, descending = self.sort_key, self.descending
679         entries = sorted(self.entries, key=lambda entry: entry.name.casefold(), reverse=descending and key == 'name')
680         if key != 'name':
681             entries.sort(key=lambda entry: getattr(entry, key), reverse=descending)
682         entries.sort(key=lambda entry: not entry.is_dir)
683         has_parent = os.path.dirname(self.directory) != self.directory
684         self.items = ([None] if has_parent else []) + entries
685         wanted = list(select) or ([current.name] if current else [])
686         self.cursor = 0
687         for index, entry in enumerate(self.items):
688             if entry is not None and entry.name in wanted:
689                 self.cursor = index
690                 break
691         self.redraw()
692
```

- 659: открыть каталог; `select` — имена, на которые поставить курсор.
- 660–664: `fileops.list_directory` читает каталог; ошибка — окно ошибки, каталог не меняется.
- 665–668: если каталог другой — сбрасываем отметки, сообщение и строки.
- 669–670: запоминаем каталог и его объекты.
- 671: из отметок оставляем только существующие файлы (`&=` — пересечение множеств).
- 672: путь в заголовке окна.
- 673–674: заполнить список; успех.
- 675: пустая строка.
- 676: `fill_items` — упорядочить объекты и поставить курсор.

- 677: что было под курсором — чтобы курсор не «прыгнул» после пересортировки.
- 678: текущая сортировка.
- 679: сначала по имени без учёта регистра (`casefold`), по убыванию — только если выбрана сортировка по имени.
- 680–681: затем по размеру или дате. Сортировка в Python устойчивая: при равных размерах сохраняется порядок по имени.
- 682: и последней — каталоги вперёд (`not entry.is_dir` у каталога False, а False < True).
- 683–684: в начало — строка «..», если это не корень диска.
- 685: на какие имена ставить курсор: переданные или прежнее.
- 686–690: курсор на первую строку с нужным именем, иначе на нулевую.
- 691: перерисовать.
- 692: пустая строка.

Строки 693–706. Что под курсором и что отмечено

```
693      def cursor_entry(self):
694          if 0 <= self.cursor < len(self.items):
695              return self.items[self.cursor]
696          return None
697
698      def cursor_file(self):
699          entry = self.cursor_entry()
700          return entry if entry is not None and not entry.is_dir else None
701
702      def marked_files(self) -> list:
703          return [entry for entry in self.items if entry is not None and entry.name in self.marked]
704
705      # --- действия (клавиши, строка F-клавиш и меню вызывают одни и те же методы) ---
706
```

- 693–696: `cursor_entry` — объект под курсором (или `None` для «..» и пустого списка).
- 697: пустая строка.
- 698–700: `cursor_file` — то же, но только если это файл: каталоги не переименовываются.
- 701: пустая строка.
- 702–703: `marked_files` — отмеченные файлы в порядке панели.
- 704: пустая строка.
- 705–706: комментарий-раздел: F-клавиши, меню и строка клавиш вызывают одни и те же методы ниже; пустая строка.

Строки 707–776. Действия программы

```
707     def help(self):
708         self.show_dialog(MessageDialog(self, 'Как пользоваться', HELP_LINES))
709 
710     def about(self):
711         self.show_dialog(MessageDialog(self, 'О программе', [
712             'Управление файлами', '',
713             'Лабораторная работа № 3, вариант 1: переименование файла.',
714             'Парадигма меню без стандартных виджетов: всё нарисовано на холсте.']))
715 
716     def rename_one(self):
717         entry = self.cursor_file()
718         if entry is None:
719             self.message = 'Поставьте курсор на файл. Каталоги программа не переименовывает.'
720             self.redraw()
721             return
722         self.show_dialog(RenameDialog(self, entry))
723 
724     def rename_group(self):
725         files = self.marked_files()
726         if not files:
727             self.message = 'Отметьте файлы: Insert, Ctrl+A или F4 — по маске.'
728             self.redraw()
729             return
730         self.show_dialog(GroupDialog(self, files))
731 
732     def select_by_mask(self):
733         self.show_dialog(MaskDialog(self))
734 
735     def refresh(self):
736         self.open_directory(self.directory)
737 
738     def go_to_path(self):
739         self.show_dialog(PathDialog(self))
740 
741     def go_up(self):
742         parent = os.path.dirname(self.directory)
743         if parent != self.directory:
744             self.open_directory(parent, [os.path.basename(self.directory)])
745 
746     def enter(self):
747         if not self.items:
748             return
749         entry = self.cursor_entry()
750         if entry is None:
751             self.go_up()
752         elif entry.is_dir:
753             self.open_directory(entry.path)
754         else:
755             self.rename_one()
756 
757     def next_sort(self):
758         keys = [key for key, _ in SORT_KEYS]
759         self.set_sort(keys[(keys.index(self.sort_key) + 1) % len(keys)])
760 
761     def set_sort(self, key: str):
762         self.sort_key = key
763         self.fill_items()
764 
765     def toggle_descending(self):
766         self.descending = not self.descending
767         self.fill_items()
768 
769     def open_menu(self):
770         self.menu.open(0)
771         self.redraw()
772 
773     def quit(self):
774         self.alive = False
775         self.root.destroy()
776 
```

- 707–708: `help` — окно справки (F1).
- 709: пустая строка.
- 710–714: `about` — «О программе».
- 715: пустая строка.
- 716–722: `rename_one` (F2): если под курсором не файл — сообщение в строке состояния; иначе окно переименования.

- 723: пустая строка.
- 724–730: `rename_group` (F3): если ничего не отмечено — подсказка; иначе окно группы с отмеченными файлами.
- 731: пустая строка.
- 732–733: `select_by_mask` (F4) — окно маски.
- 734: пустая строка.
- 735–736: `refresh` (F5) — перечитать каталог.
- 737: пустая строка.
- 738–739: `go_to_path` (F6) — окно пути.
- 740: пустая строка.
- 741–744: `go_up` (Backspace): `os.path.dirname` даёт родительский каталог; у корня диска родитель равен самому каталогу — тогда ничего. Курсор ставится на каталог, из которого вышли.
- 745: пустая строка.
- 746–755: `enter` (Enter): «..» — вверх, каталог — войти, файл — переименовать.
- 756: пустая строка.
- 757–759: `next_sort` (F7) — следующая сортировка по кругу.
- 760: пустая строка.
- 761–763: `set_sort` — задать сортировку и пересобрать список.
- 764: пустая строка.
- 765–767: `toggle_descending` — по возрастанию или по убыванию.
- 768: пустая строка.
- 769–771: `open_menu` (F9) — открыть первое меню.
- 772: пустая строка.
- 773–775: `quit` (F10): отметить, что окна больше нет, и закрыть его (`destroy`).
- 776: пустая строка.

Строки 777–801. Отметка файлов

```
777     def toggle_mark(self, move: bool = True):
778         entry = self.cursor_file()
779         if entry is not None:
780             self.marked ^= {entry.name}
781         if move:
782             self.move_cursor(1)
783         self.redraw()
784 
785     def mark_all(self):
786         self.marked = {entry.name for entry in self.entries if not entry.is_dir}
787         self.redraw()
788 
789     def mark_entries(self, entries: list):
790         self.marked = {entry.name for entry in entries}
791         names = [entry.name for entry in entries]
792         for index, entry in enumerate(self.items):
793             if entry is not None and entry.name in names:
794                 self.cursor = index
795                 break
796         self.redraw()
797 
798     def clear_marks(self):
799         self.marked = set()
800         self.redraw()
801 
```

- 777–783: `toggle_mark` (Insert, пробел): имя файла добавляется в множество или убирается из него (`^=` — симметрическая разность: было — убрать, не было — добавить); курсор вниз, как в Norton Commander.
- 784: пустая строка.
- 785–787: `mark_all` (Ctrl+A) — отметить все файлы каталога.
- 788: пустая строка.
- 789–796: `mark_entries` — отметить файлы, найденные по маске, и поставить курсор на первый из них.
- 797: пустая строка.

- 798–800: `clear_marks` (Esc) — снять все отметки.
- 801: пустая строка.

Строки 802–836. App.apply — выполнить переименование; окна; курсор

```
802     def apply(self, changes: list):
803         try:
804             fileops.apply_plan(self.directory, changes)
805         except FileError as error:
806             self.refresh()
807             self.show_error(str(error))
808             return
809         renamed_marks = {step.new for step in changes if step.old in self.marked}
810         self.marked = (self.marked - {step.old for step in changes}) | renamed_marks
811         self.entries = fileops.list_directory(self.directory)
812         self.message = fileops.renamed_text(len(changes))
813         if len(changes) == 1:
814             self.message += ': ' + changes[0].old + ' → ' + changes[0].new
815         self.message += '.'
816         self.fill_items([step.new for step in changes])
817
818     def show_dialog(self, dialog: Dialog):
819         self.menu.close()
820         self.dialogs.append(dialog)
821         self.redraw()
822
823     def close_dialog(self, dialog: Dialog):
824         if dialog in self.dialogs:
825             self.dialogs.remove(dialog)
826         self.redraw()
827
828     def show_error(self, text: str):
829         self.show_dialog(MessageDialog(self, 'Ошибка', [text], error=True))
830
831     def move_cursor(self, step: int):
832         if self.items:
833             self.cursor = min(max(self.cursor + step, 0), len(self.items) - 1)
834
835     # --- события ---
836
```

- 802–808: `fileops.apply_plan` переименовывает файлы на диске. Если сбой — список перечитывается и показывается ошибка (уже переименованным `apply_plan` сам вернул прежние имена).
- 809–810: отметки переносятся на новые имена.
- 811: каталог перечитывается.
- 812–815: сообщение «Переименовано N файлов», для одного файла — с именами «старое → новое».
- 816: список заполняется заново, курсор — на переименованный файл.
- 817: пустая строка.
- 818–821: `show_dialog` — закрыть меню, положить окно наверх стопки, перерисовать.
- 822: пустая строка.
- 823–826: `close_dialog` — убрать окно из стопки.
- 827: пустая строка.
- 828–829: `show_error` — окно ошибки.
- 830: пустая строка.
- 831–833: `move_cursor` — сдвинуть курсор, не выходя за начало и конец списка.
- 834: пустая строка.
- 835–836: комментарий-раздел; пустая строка.

Строки 837–869. App.on_key — куда идёт нажатая клавиша

```
837     def on_key(self, event):
838         keysym = event.keysym
839         if self.dialogs:
840             self.dialogs[-1].key(event)
841         elif self.menu.opened is not None:
842             self.menu.key(event)
843         elif keysym in FUNCTION_KEYS:
844             getattr(self, FUNCTION_KEYS[keysym])()
845         elif keysym == 'Up':
846             self.move_cursor(-1)
847         elif keysym == 'Down':
848             self.move_cursor(1)
849         elif keysym == 'Prior':
850             self.move_cursor(-self.page_rows())
851         elif keysym == 'Next':
852             self.move_cursor(self.page_rows())
853         elif keysym == 'Home':
854             self.cursor = 0
855         elif keysym == 'End':
856             self.cursor = max(len(self.items) - 1, 0)
857         elif keysym in ('Return', 'KP_Enter'):
858             self.enter()
859         elif keysym == 'BackSpace':
860             self.go_up()
861         elif keysym in ('Insert', 'space'):
862             self.toggle_mark()
863         elif keysym == 'Escape':
864             self.clear_marks()
865         elif event.state & CONTROL_MASK and event.keycode == CONTROL_A:
866             self.mark_all()
867         self.redraw()
868         return 'break'             # клавиша обработана: системная реакция (например, на F10) не нужна
869
```

- 837: вызывается Tkinter при каждом нажатии; `event` — описание события.
- 838: имя клавиши.
- 839–840: если открыто окно — клавиша уходит только верхнему окну.
- 841–842: иначе, если открыто меню, — меню.
- 843–844: иначе F-клавиша: по словарю `FUNCTION_KEYS` получаем имя метода, `getattr(self, имя)` находит метод по имени, `()` вызывает его.
- 845–848: ↑ и ↓ — курсор на строку.
- 849–852: PgUp и PgDn (в Tkinter `Prior` и `Next`) — на страницу.
- 853–856: Home и End — в начало и конец списка.
- 857–858: Enter.
- 859–860: Backspace — вверх.
- 861–862: Insert или пробел — отметить.
- 863–864: Esc — снять отметки.
- 865–866: Ctrl+A по коду клавиши — отметить все.
- 867: после любой клавиши экран перерисовывается.
- 868: `return 'break'` — сказать Tkinter, что клавиша обработана, и не передавать её дальше. Иначе, например, F10 в Windows активировала бы системное меню окна.
- 869: пустая строка.

Строки 870–886. Мышь, колесо, размер окна

```
870     def on_mouse(self, event, kind: str):
871         self.canvas.focus_set()
872         self.screen.hit(event.x, event.y, kind)
873         self.redraw()
874         return 'break'
875 
876     def on_wheel(self, event):
877         if not self.dialogs and self.menu.opened is None:
878             self.move_cursor(-3 if event.delta > 0 else 3)
879             self.redraw()
880 
881     def on_resize(self, event):
882         self.screen.resize(event.width, event.height)
883         self.redraw()
884 
885     # --- рисование ---
886 
```

- 870–874: `on_mouse`: фокус на холст, `Screen.hit` находит область под курсором и вызывает её обработчик, затем перерисовка.
- 875: пустая строка.
- 876–879: колесо прокручивает список на 3 строки, если не открыты окно и меню. `event.delta > 0` — колесо от себя.
- 880: пустая строка.
- 881–883: окно изменило размер — пересчитать сетку и перерисовать.
- 884–886: пустая строка, комментарий-раздел, пустая строка.

Строки 887–905. App.redraw — перерисовка всего экрана

```
887     def page_rows(self) -> int:
888         return max(self.screen.rows - 9, 1)
889 
890     def redraw(self):
891         if not self.alive:
892             return
893         screen = self.screen
894         screen.clear(BLUE)
895         self.draw_panel(screen)
896         self.draw_status(screen)
897         self.draw_keybar(screen)
898         self.menu.draw_bar(screen)
899         if self.menu.opened is not None:
900             self.menu.draw_drop(screen)
901         for dialog in self.dialogs:
902             if dialog is self.dialogs[-1]:
903                 screen.regions = []        # мышь получает только верхнее окно
904             dialog.draw(screen)
905 
```

- 887–888: `page_rows` — сколько строк списка помещается на экране (для PgUp/PgDn).
- 889: пустая строка.
- 890: `redraw` вызывается после каждого события: экран рисуется заново целиком по состоянию программы.
- 891–892: если окна уже нет (после выхода) — ничего.
- 893–894: очистить холст и залить синим.
- 895–897: панель файлов, строка сообщений, строка F-клавиш.
- 898: строка меню.
- 899–900: выпадающее меню, если открыто, — поверх панели.
- 901–904: открытые окна по порядку. Перед верхним окном список областей мыши очищается — щелчки получает только оно (модальность).
- 905: пустая строка.

Строки 906–929. App.draw_panel: рамка, заголовки столбцов, путь

```
906      def draw_panel(self, screen: Screen):
907          cols, rows = screen.cols, screen.rows
908          top, height = 1, rows - 3
909          screen.frame(0, top, cols, height, LIGHT_CYAN, BLUE, DOUBLE)
910          size_w, date_w = 10, 16
911          name_w = cols - size_w - date_w - 4
912          size_x, date_x = 1 + name_w + 1, 1 + name_w + 1 + size_w + 1
913          arrow = ' ▼' if self.descending else ' ▲'
914          header = [('Имя', 'name'), ('Размер', 'size'), ('Изменён', 'modified')]
915          for (title, key), x, width in zip(header, (1, size_x, date_x), (name_w, size_w, date_w)):
916              title += arrow if key == self.sort_key else ''
917              screen.text(x + (width - len(title)) // 2, top + 1, title, YELLOW)
918              screen.region(x, top + 1, width, 1, lambda _x, _y, key=key: self.click_header(key))
919          for x in (size_x - 1, date_x - 1):
920              screen.text(x, top, '╤', LIGHT_CYAN)
921              for row in range(top + 1, top + height - 3):
922                  screen.text(x, row, '│', LIGHT_CYAN)
923              screen.text(x, top + height - 3, '┴', LIGHT_CYAN)
924          screen.text(0, top + height - 3, '╟' + '─' * (cols - 2) + '╢', LIGHT_CYAN)
925          for x in (size_x - 1, date_x - 1):
926              screen.text(x, top + height - 3, '┴', LIGHT_CYAN)
927          title = self.directory if len(self.directory) <= cols - 8 else '…' + self.directory[-(cols - 9):]
928          screen.text((cols - len(title) - 2) // 2, top, ' ' + title + ' ', BLACK, CYAN)
929
```

- 906: нарисовать панель файлов.
- 907–908: размеры экрана; панель — со 2-й строки, высотой на 3 строки меньше экрана (меню сверху, сообщения и F-строка снизу).
- 909: двойная рамка панели.
- 910–912: ширины столбцов: размер 10, дата 16, имя — всё остальное; где начинаются столбцы.
- 913: стрелка направления сортировки.
- 914–918: заголовки «Имя», «Размер», «Изменён» жёлтым по центру столбцов; у текущей сортировки — стрелка; щелчок по заголовку меняет сортировку.
- 919–923: вертикальные разделители столбцов с переходами в рамку «╤» сверху и «┴» снизу.
- 924–926: горизонтальная линия над строкой итогов «╟──╢».
- 927–928: путь каталога в верхней линии рамки (длинный — с «…» в начале).
- 929: пустая строка.

Строки 930–966. App.draw_panel: строки файлов и итоги

```
930         first, visible = top + 2, height - 5
931         if self.cursor < self.top:
932             self.top = self.cursor
933         if self.cursor >= self.top + visible:
934             self.top = self.cursor - visible + 1
935         self.top = max(min(self.top, max(len(self.items) - visible, 0)), 0)
936         for row in range(visible):
937             index = self.top + row
938             if index >= len(self.items):
939                 break
940             entry = self.items[index]
941             y = first + row
942             if entry is None:
943                 name, size, date, marked = '..', 'Вверх', '', False
944             else:
945                 name = entry.name.upper() if entry.is_dir else entry.name
946                 size = '<Каталог>' if entry.is_dir else fileops.format_size(entry.size)
947                 date = fileops.format_date(entry.modified)
948                 marked = entry.name in self.marked
949             fg = YELLOW if marked else (WHITE if entry is None or entry.is_dir else LIGHT_CYAN)
950             if index == self.cursor:
951                 fg, bg = (YELLOW if marked else BLACK), CYAN
952                 screen.text(1, y, '', fg, bg, cols - 2)
953             screen.text(1, y, (('√' if marked else ' ') + name), fg, width=name_w)
954             screen.text(size_x, y, size.rjust(size_w), fg)
955             screen.text(date_x, y, date.ljust(date_w), fg)
956             screen.region(1, y, cols - 2, 1, lambda _x, _y, index=index: self.click_row(index),
957                           lambda _x, _y, index=index: self.double_row(index),
958                           lambda _x, _y, index=index: self.right_row(index))
959
960         info = 'Всего: ' + fileops.entries_text(self.entries)
961         marked = self.marked_files()
962         if marked:
963             info += '   Отмечено: ' + fileops.files_text(len(marked)) + ', ' + \
964                     fileops.format_size(sum(entry.size for entry in marked))
965         screen.text(1, top + height - 2, info, YELLOW if marked else LIGHT_CYAN, width=cols - 2)
966
```

- 930: первая строка списка и число видимых строк.
- 931–935: прокрутка: если курсор выше или ниже видимой части — сдвигаем `top`; не даём прокрутить за конец.
- 936–939: перебор видимых строк; список кончился — выход.
- 940–941: объект и экранная строка.
- 942–943: строка «..» — «Вверх» в столбце размера.
- 944–948: обычный объект: каталоги — заглавными, размер — `<Каталог>` или «47,2 КБ» (`format_size`), дата (`format_date`), отмечен ли.
- 949: цвет: отмеченный — жёлтый, каталог и «..» — белый, файл — светло-бирюзовый.
- 950–952: строка под курсором — бирюзовая полоса во всю ширину, текст чёрный (отмеченный — жёлтый).
- 953–955: имя (с «√» у отмеченных), размер прижат вправо (`rjust`), дата.
- 956–958: область строки: щелчок ставит курсор, двойной — Enter, правой кнопкой — отметка. `index=index` запоминает номер строки.
- 959: пустая строка.
- 960–965: строка итогов: «Всего: 3 каталога, 12 файлов», при отметках — «Отмечено: N файлов, размер» жёлтым.
- 966: пустая строка.

Строки 967–986. Строка сообщений и строка F-клавиш

```
967     def draw_status(self, screen: Screen):
968         y = screen.rows - 2
969         text = self.message or (self.directory + '>')
970         screen.text(0, y, text, WHITE if self.message else '#c0c0c0', BLACK, screen.cols)
971
972     def draw_keybar(self, screen: Screen):
973         y = screen.rows - 1
974         screen.text(0, y, '', BLACK, BLACK, screen.cols)
975         width = screen.cols // len(KEYBAR)
976         for index, (number, label, action) in enumerate(KEYBAR):
977             x = index * width
978             key = 'F' + number                # в узком окне сокращается подпись, а не название клавиши
979             screen.text(x, y, key, WHITE)
980             enabled = action is not None
981             screen.text(x + len(key), y, label, BLACK if enabled else DARK_GRAY, CYAN, width - len(key) - 1)
982             if enabled:
983                 screen.region(x, y, width - 1, 1, lambda _x, _y, action=action: getattr(self, action)())
984
985     # --- мышь в панели ---
986
```

- 967–970: `draw_status`: предпоследняя строка экрана — сообщение (белым) или путь с «>», как приглашение командной строки DOS (серым).
- 971: пустая строка.
- 972: `draw_keybar` — последняя строка.
- 973–974: строка и чёрный фон.
- 975: ширина ячейки — экран делится на 10 равных частей.
- 976–977: перебор клавиш из `KEYBAR`, левый край ячейки.
- 978–979: «F» и номер белым на чёрном.
- 980–981: подпись чёрным на бирюзовом (у пустой F8 — серым). Если места мало, `text` обрежет подпись, а «F1»–«F10» останутся целыми.
- 982–983: у клавиши с действием — область: щелчок вызывает тот же метод, что и клавиша.
- 984–986: пустая строка, комментарий-раздел, пустая строка.

Строки 987–1004. Мышь в панели

```
987     def click_header(self, key: str):
988         if self.sort_key == key:
989             self.toggle_descending()
990         else:
991             self.set_sort(key)
992
993     def click_row(self, index: int):
994         self.cursor = index
995
996     def double_row(self, index: int):
997         self.cursor = index
998         self.enter()
999
1000    def right_row(self, index: int):
1001        self.cursor = index
1002        self.toggle_mark(move=False)
1003
1004
```

- 987–991: щелчок по заголовку: тот же столбец — сменить направление, другой — сортировать по нему.
- 992: пустая строка.
- 993–994: щелчок по строке — курсор на неё.
- 995: пустая строка.
- 996–998: двойной щелчок — курсор и Enter.
- 999: пустая строка.
- 1000–1002: правая кнопка — курсор и отметка без сдвига курсора.
- 1003–1004: пустые строки.

Строки 1005–1024. main — запуск программы

```
1005  def main() -> int:
1006      try:
1007          ctypes.windll.shcore.SetProcessDpiAwareness(1)
1008      except (AttributeError, OSError):
1009          pass
1010      directory = os.path.realpath(os.getcwd())
1011      if len(sys.argv) > 1:
1012          try:
1013              directory = fileops.resolve_directory(directory, sys.argv[1])
1014          except FileError as error:
1015              print(error)
1016              return 1
1017      root = tk.Tk()
1018      App(root, directory)
1019      root.mainloop()
1020      return 0
1021
1022
1023  if __name__ == '__main__':
1024      sys.exit(main())
```

- 1005: `main` — точка входа; возвращает код завершения (0 — успех).
- 1006–1009: просим Windows не растягивать окно на экранах с масштабом, а рисовать чётко. На других системах нет `windll` — ошибка `AttributeError` перехватывается и пропускается.
- 1010: каталог по умолчанию — тот, из которого запущена программа.
- 1011–1016: если в командной строке указан каталог (`python nc_menu.py demo`), он проверяется; если его нет — сообщение и выход с кодом 1.
- 1017: создать окно.
- 1018: создать приложение — оно строит холст, привязывает события и открывает каталог.
- 1019: `mainloop` — главный цикл Tkinter: ждёт событий (клавиши, мышь) и вызывает наши обработчики, пока окно не закроют.
- 1020: окно закрыто — успешное завершение.
- 1021–1022: пустые строки.
- 1023–1024: `if __name__ == '__main__'` — выполняется, только если файл запущен напрямую, а не импортирован. Тогда вызываем `main()` и передаём код завершения системе.

5. Построчный разбор fileops.py

Строки 1–7. Описание и подключение модулей

```
 1  """Общая часть обеих программ: чтение каталога, маски имён, переименование файлов."""
 2
 3  import os
 4  import re
 5  from dataclasses import dataclass
 6  from datetime import datetime
 7
```

- 1: строка документации: общая часть программ — каталог, маски, переименование. Интерфейса в этом файле нет, поэтому им пользуются все три программы (ЛР2 и ЛР3).
- 2: пустая строка.
- 3: `os` — файлы и пути.
- 4: `re` — регулярные выражения: по ним проверяются маски.
- 5: `dataclass` — декоратор, который сам создаёт конструктор и сравнение для простых классов-записей.
- 6: `datetime` — дата и время изменения файла.
- 7: пустая строка.

Строки 8–14. Ограничения на имена файлов в Windows

```
 8  FORBIDDEN_CHARS = '\\/:*?"<>|'
 9  RESERVED_NAMES = {'CON', 'PRN', 'AUX', 'NUL'} | {
10      port + str(number) for port in ('COM', 'LPT') for number in range(1, 10)
11  }
12  MAX_NAME_LENGTH = 255
13
14
```

- 8: символы, запрещённые в именах файлов Windows. `\\` в строке — один обратный слеш.
- 9–11: зарезервированные имена устройств (CON, PRN, AUX, NUL, COM1–COM9, LPT1–LPT9): файл с таким именем Windows создать не даст. `|` объединяет два множества; второе построено генератором множества.
- 12: максимальная длина имени.
- 13–14: пустые строки.

Строки 15–33. Классы FileError, Entry, Rename

```
15  class FileError(Exception):
16      """Ошибка, текст которой показывается пользователю без изменений."""
17
18
19  @dataclass(frozen=True)
20  class Entry:
21      name: str
22      path: str
23      is_dir: bool
24      size: int
25      modified: datetime
26
27
28  @dataclass(frozen=True)
29  class Rename:
30      old: str
31      new: str
32
33
```

- 15–16: `FileError` — своя ошибка, наследник `Exception`. Её текст показывается пользователю как есть. Тело класса — только строка документации.
- 17–18: пустые строки.
- 19–25: `Entry` — объект каталога: имя, полный путь, каталог ли, размер, время изменения. `@dataclass` создаёт конструктор `Entry(name, path, is_dir, size, modified)`; `frozen=True` — поля нельзя менять после создания.
- 26–27: пустые строки.
- 28–31: `Rename` — один шаг переименования: старое и новое имя. Список таких шагов — план.

- 32–33: пустые строки.

Строки 34–57. list_directory — прочитать каталог

```
34  # --- каталог -----------------------------------------------------------
35
36  def list_directory(directory: str) -> list[Entry]:
37      entries = []
38      try:
39          with os.scandir(directory) as scan:
40              for item in scan:
41                  try:
42                      info = item.stat()
43                      is_dir = item.is_dir()
44                      modified = datetime.fromtimestamp(info.st_mtime)
45                  except (OSError, ValueError, OverflowError):
46                      continue
47                  entries.append(Entry(item.name, item.path, is_dir, 0 if is_dir else info.st_size, modified))
48      except FileNotFoundError:
49          raise FileError('Каталог не найден: ' + directory) from None
50      except PermissionError:
51          raise FileError('Нет доступа к каталогу: ' + directory) from None
52      except OSError as error:
53          raise FileError('Не удалось прочитать каталог: ' + (error.strerror or str(error))) from None
54      entries.sort(key=lambda entry: (not entry.is_dir, entry.name.casefold()))
55      return entries
56
57
```

- 34: комментарий-раздел.
- 35: пустая строка.
- 36: функция возвращает список `Entry`. `-> list[Entry]` — подсказка типа результата.
- 37: пустой список.
- 38–39: `os.scandir` открывает каталог для перебора; `with` гарантирует, что он будет закрыт.
- 40: перебор объектов каталога.
- 41–46: у каждого объекта берём сведения (`stat`), каталог ли он и время изменения. Если объект нельзя прочитать (нет доступа, битая дата) — пропускаем его (`continue`), а не падаем.
- 47: добавляем `Entry`; у каталогов размер 0.
- 48–53: ошибки чтения самого каталога превращаются в `FileError` с понятным текстом: не найден, нет доступа, другая ошибка. `from None` скрывает техническую цепочку ошибок.
- 54: сортировка: сначала каталоги (`not is_dir` у каталога False), затем по имени без учёта регистра.
- 55: результат.
- 56–57: пустые строки.

Строки 58–80. resolve_directory и find_entry

```
58  def resolve_directory(current: str, target: str) -> str:
59      """Путь каталога по тому, что ввёл пользователь: относительный или абсолютный."""
60      if len(target) == 2 and target[1] == ':':
61          target += os.sep
62      path = os.path.normpath(os.path.join(current, target))
63      if not os.path.exists(path):
64          raise FileError('Каталог не найден: ' + target)
65      if not os.path.isdir(path):
66          raise FileError('Это файл, а не каталог: ' + target)
67      try:
68          os.scandir(path).close()
69      except OSError:
70          raise FileError('Нет доступа к каталогу: ' + target) from None
71      return os.path.realpath(path)
72
73
74  def find_entry(entries: list[Entry], name: str) -> Entry | None:
75      for entry in entries:
76          if entry.name.casefold() == name.casefold():
77              return entry
78      return None
79
80
```

- 58–59: из введённого пользователем пути получить настоящий каталог.

- 60–61: «D:» превращается в «D:\», то есть в корень диска.
- 62: `os.path.join(current, target)` склеивает пути; если `target` абсолютный (`C:\Windows`), текущий каталог отбрасывается. `normpath` убирает `..` и лишние слеши. Так одна строка обрабатывает и относительный, и абсолютный путь.
- 63–66: нет такого пути или это файл — ошибки из таблицы варианта.
- 67–70: пробуем открыть каталог: нет прав — ошибка.
- 71: `realpath` — окончательный путь без символических ссылок.
- 72–73: пустые строки.
- 74–78: `find_entry` — найти объект по имени без учёта регистра (в Windows «A.TXT» и «a.txt» — один файл). `Entry | None` — вернётся объект или `None`.
- 79–80: пустые строки.

Строки 81–107. Маски: is_mask, mask_regex, matches, select

```
 81  # --- маски ----------------------------------------------------------------
 82
 83  def is_mask(text: str) -> bool:
 84      return '*' in text or '?' in text
 85
 86
 87  def mask_regex(mask: str) -> re.Pattern:
 88      parts = []
 89      for char in mask:
 90          if char == '*':
 91              parts.append('(.*)')
 92          elif char == '?':
 93              parts.append('(.)')
 94          else:
 95              parts.append(re.escape(char))
 96      return re.compile(''.join(parts), re.IGNORECASE | re.DOTALL)
 97
 98
 99  def matches(name: str, mask: str) -> bool:
100      return mask_regex(mask).fullmatch(name) is not None
101
102
103  def select(entries: list[Entry], mask: str, files_only: bool = False) -> list[Entry]:
104      return [entry for entry in entries
105              if matches(entry.name, mask) and not (files_only and entry.is_dir)]
106
107
```

- 81–82: комментарий-раздел и пустая строка.
- 83–84: `is_mask` — есть ли в строке `*` или `?`.
- 85–86: пустые строки.
- 87: `mask_regex` — перевести маску в регулярное выражение.
- 88–95: идём по маске посимвольно: `*` → группа `(.*)` (любые символы, запомнить), `?` → `(.)` (один символ, запомнить), остальное — как есть; `re.escape` экранирует символы, особые для регулярных выражений (например, точку).
- 96: склеиваем части и компилируем: без учёта регистра, точка совпадает с любым символом.
- 97–98: пустые строки.
- 99–100: `matches` — подходит ли имя под маску целиком (`fullmatch`).
- 101–102: пустые строки.
- 103–105: `select` — объекты, подходящие под маску; при `files_only` — без каталогов.
- 106–107: пустые строки.

Строки 108–122. substitute — новое имя по маске (для команды ren)

```
108  def substitute(name: str, mask: str, new_mask: str) -> str:
109      """Имя по новой маске: каждый знак * и ? в ней заменяется тем, что совпало
110      с таким же по счёту знаком исходной маски."""
111      found = mask_regex(mask).fullmatch(name)
112      wildcards = [char for char in mask if char in '*?']
113      captured = {'*': [], '?': []}
114      for char, text in zip(wildcards, found.groups()):
115          captured[char].append(text)
116      stars, marks = iter(captured['*']), iter(captured['?'])
117      return ''.join(next(stars) if char == '*' else next(marks) if char == '?' else char
118                     for char in new_mask)
119
120
121  # --- план переименования -----------------------------------------------
122
```

- 108–110: заменить в новой маске каждый * и ? тем, что совпало с таким же по счёту знаком исходной маски.
- 111: сопоставляем имя с выражением исходной маски; группы хранят совпавшие куски.
- 112: какие знаки были в исходной маске по порядку.
- 113–115: раскладываем совпавшие куски по двум спискам — для звёздочек и для вопросов.
- 116: итераторы: `next` будет выдавать куски по одному.
- 117–118: идём по новой маске: * → следующий кусок звёздочки, ? → следующий кусок вопроса, остальное — как есть; `join` склеивает. Пример: `IMG_0001.jpg`, `IMG_*.jpg` → `отпуск_*.jpg` даёт `отпуск_0001.jpg`.
- 119–120: пустые строки.
- 121–122: комментарий-раздел и пустая строка.

Строки 123–145. plan_rename — план команды ren

```
123  def plan_rename(entries: list[Entry], source: str, target: str) -> list[Rename]:
124      """План команды ren: имя и новое имя либо маска и новая маска."""
125      for text in (source, target):
126          if '\\' in text or '/' in text:
127              raise FileError('Имя указывается без пути, команда работает в текущем каталоге: ' + text)
128      if not is_mask(source):
129          entry = find_entry(entries, source)
130          if entry is None:
131              raise FileError('Файл не найден: ' + source)
132          if entry.is_dir:
133              raise FileError('Это каталог, а не файл: ' + entry.name)
134          if is_mask(target):
135              raise FileError('Знаки * и ? в новом имени допустимы только вместе с маской: ' + target)
136          return [Rename(entry.name, target)]
137      for char in '*?':
138          if target.count(char) > source.count(char):
139              raise FileError('В новой маске знаков ' + char + ' больше, чем в исходной: ' + target)
140      files = select(entries, source, files_only=True)
141      if not files:
142          raise FileError('Нет файлов, соответствующих маске: ' + source)
143      return [Rename(entry.name, substitute(entry.name, source, target)) for entry in files]
144
145
```

- 123–124: план для командного языка (ЛР2): либо имя и новое имя, либо маска и новая маска. В ЛР3 не используется, но оставлен — модуль общий.
- 125–127: имена указываются без пути.
- 128–136: одно имя: файл должен существовать и быть файлом; в новом имени нельзя * и ?; план из одного шага.
- 137–139: в новой маске знаков * и ? не больше, чем в исходной.
- 140–142: файлы под маску; если нет — ошибка из таблицы варианта.
- 143: план — по шагу на файл, новое имя через `substitute`.
- 144–145: пустые строки.

Строки 146–160. Три действия окна группы

```
146  def add_prefix(name: str, text: str) -> str:
147      return text + name
148
149
150  def add_suffix(name: str, text: str) -> str:
151      base, extension = os.path.splitext(name)
152      return base + text + extension
153
154
155  def change_extension(name: str, extension: str) -> str:
156      extension = extension.strip().lstrip('.')
157      base = os.path.splitext(name)[0]
158      return base + '.' + extension if extension else base
159
160
```

- 146–147: `add_prefix` — текст в начало: `old_` + `readme.txt`.
- 148–149: пустые строки.
- 150–152: `add_suffix` — текст в конец перед расширением: `os.path.splitext` делит имя на основу и расширение, текст вставляется между ними.
- 153–154: пустые строки.
- 155–158: `change_extension` — новое расширение: лишние точка и пробелы убираются; пустое расширение — имя без расширения.
- 159–160: пустые строки.

Строки 161–176. validate_name — допустимо ли имя

```
161  def validate_name(name: str) -> None:
162      if name == '':
163          raise FileError('Новое имя не задано.')
164      if any(char in FORBIDDEN_CHARS or ord(char) < 32 for char in name):
165          problem = 'Нельзя использовать символы \\ / : * ? " < > |'
166      elif name[-1] in ' .':
167          problem = 'Имя не может заканчиваться точкой или пробелом.'
168      elif name.split('.')[0].strip().upper() in RESERVED_NAMES:
169          problem = 'Это имя зарезервировано системой.'
170      elif len(name) > MAX_NAME_LENGTH:
171          problem = 'Имя длиннее ' + str(MAX_NAME_LENGTH) + ' символов.'
172      else:
173          return
174      raise FileError('Недопустимое имя файла: ' + name + '. ' + problem)
175
176
```

- 161: проверить одно новое имя.
- 162–163: пустое имя.
- 164–165: запрещённые символы или управляющие (код меньше 32). `any(...)` — истинно, если условие выполнено хотя бы для одного символа.
- 166–167: точка или пробел в конце — Windows их отбрасывает.
- 168–169: зарезервированное имя устройства.
- 170–171: слишком длинное.
- 172–173: всё в порядке — выходим.
- 174: иначе — ошибка «Недопустимое имя файла: … » с причиной.
- 175–176: пустые строки.

Строки 177–199. check_plan — проверить весь план до переименования

```
177  def check_plan(entries: list[Entry], plan: list[Rename]) -> list[Rename]:
178      """Проверяет план целиком и возвращает шаги, в которых имя действительно меняется.
179      Любая проблема отменяет всё переименование, поэтому файлы не трогаются до конца проверки."""
180      changes = [step for step in plan if step.new != step.old]
181      if not changes:
182          if len(plan) == 1:
183              raise FileError('Новое имя совпадает со старым: ' + plan[0].old)
184          raise FileError('Новые имена совпадают со старыми, переименовывать нечего.')
185      existing = {entry.name.casefold(): entry for entry in entries}
186      targets = {}
187      for step in changes:
188          validate_name(step.new)
189          key = step.new.casefold()
190          if key in targets:
191              raise FileError('Файлы ' + targets[key] + ' и ' + step.old + ' получили бы одно имя '
192                              + step.new + '. Переименование отменено.')
193          targets[key] = step.old
194          if key in existing and key != step.old.casefold():
195              kind = 'Каталог' if existing[key].is_dir else 'Файл'
196              raise FileError(kind + ' с именем ' + step.new + ' уже существует. Переименование отменено.')
197      return changes
198
199
```

- 177–179: главное правило «всё или ничего»: сначала проверяется весь план, файлы до конца проверки не трогаются.
- 180: шаги, в которых имя действительно меняется.
- 181–184: если меняться нечему — ошибка.
- 185: словарь существующих имён (без учёта регистра).
- 186: новые имена, уже занятые в этом плане.
- 187–188: каждое новое имя проверяется на допустимость.
- 189–193: два файла группы получили бы одно имя — ошибка.
- 194–196: новое имя занято другим объектом каталога — ошибка «Файл (Каталог) с именем … уже существует». Исключение — смена только регистра своего же имени.
- 197: план проверен — вернуть шаги.
- 198–199: пустые строки.

Строки 200–240. apply_plan — переименовать и откатить при сбое

```
200  def apply_plan(directory: str, changes: list[Rename]) -> None:
201      """Переименовывает файлы; при сбое возвращает прежние имена уже переименованным."""
202      done = []
203      for step in changes:
204          source = os.path.join(directory, step.old)
205          target = os.path.join(directory, step.new)
206          try:
207              if step.old.casefold() != step.new.casefold() and os.path.lexists(target):
208                  raise FileExistsError(target)
209              os.rename(source, target)
210          except OSError as error:
211              stuck = _roll_back(directory, done)
212              message = 'Не удалось переименовать ' + step.old + ': ' + _reason(error) + '. Переименование отменено.'
213              if stuck:
214                  message += ' Прежние имена не возвращены: ' + ', '.join(stuck) + '.'
215              raise FileError(message) from None
216          done.append(step)
217
218
219  def _roll_back(directory: str, done: list[Rename]) -> list[str]:
220      stuck = []
221      for step in reversed(done):
222          try:
223              os.rename(os.path.join(directory, step.new), os.path.join(directory, step.old))
224          except OSError:
225              stuck.append(step.new)
226      return stuck
227
228
229  def _reason(error: OSError) -> str:
230      if isinstance(error, FileExistsError):
231          return 'такое имя уже занято'
232      if isinstance(error, FileNotFoundError):
233          return 'файл не найден, возможно, он уже переименован или удалён'
234      if isinstance(error, PermissionError):
235          return 'нет доступа или файл открыт в другой программе'
236      return error.strerror or str(error)
237
238
239  # --- текст для вывода ----------------------------------------------------
240
```

- 200–201: выполнить проверенный план.
- 202: уже сделанные шаги.
- 203–205: полные пути старого и нового имени.
- 206–209: ещё раз проверяем, что новое имя свободно (могло появиться за это время), и переименовываем `os.rename`.
- 210–215: сбой (файл открыт в другой программе и т. п.): `_roll_back` возвращает прежние имена уже переименованным файлам, затем ошибка с причиной; если что-то вернуть не удалось — это тоже сообщается.
- 216: шаг сделан.
- 217–218: пустые строки.
- 219–226: `_roll_back` — откат в обратном порядке; имена, которые вернуть не удалось, собираются в список.
- 227–228: пустые строки.
- 229–236: `_reason` — понятная причина ошибки по её типу: имя занято, файл не найден, нет доступа, иначе системный текст.
- 237–240: пустые строки, комментарий-раздел, пустая строка.

Строки 241–256. format_size и format_date

```
241  def format_size(size: int) -> str:
242      if size < 1024:
243          return str(size) + ' Б'
244      value = float(size)
245      for unit in ('КБ', 'МБ', 'ГБ'):
246          value /= 1024
247          if value < 1024 or unit == 'ГБ':
248              break
249      text = ('%.1f' % value).replace('.', ',')
250      return (text[:-2] if text.endswith(',0') else text) + ' ' + unit
251
252
253  def format_date(moment: datetime) -> str:
254      return moment.strftime('%d.%m.%Y %H:%M')
255
256
```

- 241: размер в понятном виде.
- 242–243: меньше 1 КБ — в байтах.
- 244–248: делим на 1024, пока число не станет меньше 1024: КБ, МБ, ГБ.
- 249–250: одна цифра после запятой, запятая вместо точки, «,0» отбрасывается: «47,2 КБ», «5 МБ».
- 251–252: пустые строки.
- 253–254: дата как «14.01.2025 10:20».
- 255–256: пустые строки.

Строки 257–287. Слова в нужной форме

```
257  def plural(count: int, one: str, few: str, many: str) -> str:
258      if 11 <= count % 100 <= 14:
259          return many
260      if count % 10 == 1:
261          return one
262      if 2 <= count % 10 <= 4:
263          return few
264      return many
265
266
267  def files_text(count: int) -> str:
268      return str(count) + ' ' + plural(count, 'файл', 'файла', 'файлов')
269
270
271  def folders_text(count: int) -> str:
272      return str(count) + ' ' + plural(count, 'каталог', 'каталога', 'каталогов')
273
274
275  def entries_text(entries: list[Entry]) -> str:
276      folders = sum(1 for entry in entries if entry.is_dir)
277      files = len(entries) - folders
278      parts = []
279      if folders:
280          parts.append(folders_text(folders))
281      if files:
282          parts.append(files_text(files))
283      return ', '.join(parts) if parts else 'ничего'
284
285
286  def renamed_text(count: int) -> str:
287      return plural(count, 'Переименован ', 'Переименовано ', 'Переименовано ') + files_text(count)
```

- 257–264: `plural` — форма слова по числу: 1 файл, 2 файла, 5 файлов; 11–14 — всегда «файлов».
- 265–266: пустые строки.
- 267–268: `files_text` — «N файл(а/ов)».
- 269–270: пустые строки.
- 271–272: `folders_text` — «N каталог(а/ов)».
- 273–274: пустые строки.
- 275–283: `entries_text` — «3 каталога, 12 файлов» или «ничего».
- 284–285: пустые строки.
- 286–287: `renamed_text` — «Переименован 1 файл», «Переименовано 4 файла».

6. Построчный разбор make_demo.py

Строки 1–6. Описание и модули

```
1  """Создаёт тестовый каталог demo для проверки обеих программ."""
2
3  import os
4  import sys
5  from datetime import datetime
6
```

- 1: скрипт создаёт тестовый каталог `demo`.
- 2: пустая строка.
- 3–5: `os` — файлы, `sys` — аргументы, `datetime` — разбор дат.
- 6: пустая строка.

Строки 7–36. Что будет в тестовом каталоге

```
7  FILES = [
8      ('data_2024.csv', 48_300, '14.01.2025 10:20'),
9      ('data_2025.csv', 52_100, '12.01.2026 09:45'),
10     ('readme.md', 2_900, '03.09.2026 16:30'),
11     ('readme.txt', 2_400, '15.06.2026 12:00'),
12     ('заметки.txt', 3_200, '05.10.2026 09:12'),
13     ('отчёт_1.docx', 118_000, '20.09.2026 18:40'),
14     ('отчёт_2.docx', 96_500, '27.09.2026 19:05'),
15     ('отчёт_3.docx', 134_200, '04.10.2026 17:55'),
16     ('план.txt', 1_800, '01.10.2026 10:30'),
17     ('презентация.pptx', 620_000, '24.09.2026 21:15'),
18     ('список покупок.txt', 640, '08.10.2026 19:03'),
19     ('черновик.tmp', 0, '02.10.2026 11:47'),
20     ('Архив/отчёт_2025.docx', 87_000, '18.12.2025 14:10'),
21     ('Документы/договор.pdf', 340_000, '10.09.2026 15:20'),
22     ('Документы/резюме.docx', 64_000, '01.09.2026 11:00'),
23     ('Фото/IMG_0001.jpg', 348_000, '07.07.2026 13:45'),
24     ('Фото/IMG_0002.jpg', 312_000, '07.07.2026 13:46'),
25     ('Фото/IMG_0003.jpg', 365_000, '07.07.2026 13:48'),
26     ('Фото/IMG_0004.jpg', 331_000, '08.07.2026 09:02'),
27 ]
28
29 FOLDER_DATES = {
30     'Архив': '18.12.2025 14:10',
31     'Документы': '10.09.2026 15:20',
32     'Фото': '08.07.2026 09:02',
33 }
34
35 TEXT_EXTENSIONS = ('.txt', '.md', '.csv')
36
```

- 7–27: `FILES` — список файлов: путь внутри demo, размер в байтах, дата изменения. Имена с «/» лежат в подкаталогах (Архив, Документы, Фото). `48_300` — то же, что 48300, подчёркивание только для читаемости.
- 28: пустая строка.
- 29–33: даты изменения самих подкаталогов.
- 34: пустая строка.
- 35: какие файлы заполнять текстом, остальные — нулевыми байтами.
- 36: пустая строка.

Строки 37–60. Создание файлов

```
37
38  def set_time(path: str, moment: str) -> None:
39      timestamp = datetime.strptime(moment, '%d.%m.%Y %H:%M').timestamp()
40      os.utime(path, (timestamp, timestamp))
41
42
43  def make_file(path: str, size: int, moment: str) -> None:
44      os.makedirs(os.path.dirname(path), exist_ok=True)
45      with open(path, 'wb') as file:
46          if path.endswith(TEXT_EXTENSIONS):
47              line = ('Строка файла ' + os.path.basename(path) + '\n').encode('utf-8')
48              file.write((line * (size // len(line) + 1))[:size])
49          else:
50              file.write(bytes(size))
51      set_time(path, moment)
52
53
54  def build(target: str) -> None:
55      for name, size, moment in FILES:
56          make_file(os.path.join(target, *name.split('/')), size, moment)
57      for name, moment in FOLDER_DATES.items():
58          set_time(os.path.join(target, name), moment)
59
60
```

- 37: пустая строка.
- 38–40: `set_time` — поставить файлу или каталогу дату изменения (`os.utime`), чтобы в списке были разные даты.
- 41–42: пустые строки.
- 43–51: `make_file`: создать подкаталог, если нужно; записать текст нужной длины (строка повторяется и обрезается) или нулевые байты; поставить дату.
- 52–53: пустые строки.
- 54–58: `build` — создать все файлы и поставить даты каталогам.
- 59–60: пустые строки.

Строки 61–73. clear — очистить каталог

```
61  def clear(target: str) -> None:
62      """Удаляет содержимое каталога. Сам каталог и подкаталоги, открытые в другой программе
63      (например, в Проводнике), остаются на месте: они уже пусты и будут заполнены заново."""
64      for folder, folders, files in os.walk(target, topdown=False):
65          for name in files:
66              os.remove(os.path.join(folder, name))
67          for name in folders:
68              try:
69                  os.rmdir(os.path.join(folder, name))
70              except OSError:
71                  pass
72
73
```

- 61–63: удалить содержимое каталога. Каталоги, открытые в другой программе (например, в Проводнике), удалить нельзя — они остаются пустыми и заполняются заново.
- 64: `os.walk(..., topdown=False)` обходит дерево снизу вверх: сначала вложенное.
- 65–66: удалить файлы.
- 67–71: удалить пустые подкаталоги; если не получилось — пропустить.
- 72–73: пустые строки.

Строки 74–94. main — запуск скрипта

```
74  def main() -> int:
75      if len(sys.argv) > 1:
76          target = os.path.abspath(sys.argv[1])
77          if os.path.exists(target) and (not os.path.isdir(target) or os.listdir(target)):
78              print('Каталог уже существует и не пуст, укажите другой путь: ' + target)
79              return 1
80      else:
81          # каталог demo рядом со скриптом принадлежит ему и заполняется заново
82          target = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'demo')
83          try:
84              clear(target)
85          except OSError as error:
86              print('Не удалось очистить каталог ' + target + ': ' + (error.strerror or str(error)))
87              return 1
88      build(target)
89      print('Тестовый каталог создан: ' + target)
90      return 0
91
92
93  if __name__ == '__main__':
94      sys.exit(main())
```

- 74: точка входа.
- 75–79: если указан путь (`python make_demo.py путь`) — создать там, но только в пустом или новом каталоге, чтобы ничего не испортить.
- 80–87: без аргумента — каталог `demo` рядом со скриптом; его можно очищать, он принадлежит скрипту.
- 88–90: создать файлы, сообщить, успешный код.
- 91–92: пустые строки.
- 93–94: запуск `main()` при прямом запуске файла.

7. Построчный разбор test_fileops.py

Строки 1–11. Описание и модули

```
 1  """Тесты общего модуля. Запуск: python -m unittest test_fileops"""
 2
 3  import os
 4  import tempfile
 5  import unittest
 6  from unittest import mock
 7
 8  import fileops
 9  from fileops import FileError, Rename
10
11
```

- 1: строка документации и способ запуска: `python -m unittest test_fileops`.
- 2: пустая строка.
- 3–6: `os`, `tempfile` (временный каталог), `unittest` (тесты), `mock` (подмена функции).
- 7: пустая строка.
- 8–9: проверяемый модуль.
- 10–11: пустые строки.

Строки 12–43. DirectoryCase — общая подготовка

```
12  class DirectoryCase(unittest.TestCase):
13      FILES = ['a.txt', 'b.txt', 'b.md', 'IMG_01.jpg', 'IMG_02.jpg', 'мой файл.txt']
14      FOLDERS = ['Фото', 'a.dir']
15
16      def setUp(self):
17          temp = tempfile.TemporaryDirectory()
18          self.addCleanup(temp.cleanup)
19          self.directory = os.path.realpath(temp.name)
20          for name in self.FILES:
21              with open(os.path.join(self.directory, name), 'w', encoding='utf-8') as file:
22                  file.write(name)
23          for name in self.FOLDERS:
24              os.mkdir(os.path.join(self.directory, name))
25
26      def entries(self):
27          return fileops.list_directory(self.directory)
28
29      def names(self):
30          return sorted(os.listdir(self.directory))
31
32      def rename(self, source, target):
33          entries = self.entries()
34          changes = fileops.check_plan(entries, fileops.plan_rename(entries, source, target))
35          fileops.apply_plan(self.directory, changes)
36          return changes
37
38      def assert_error(self, text, function, *arguments):
39          with self.assertRaises(FileError) as caught:
40              function(*arguments)
41          self.assertEqual(str(caught.exception), text)
42
43
```

- 12: класс-основа для тестов с настоящим каталогом.
- 13–14: какие файлы и каталоги создать.
- 15: пустая строка.
- 16–24: `setUp` выполняется перед каждым тестом: создаётся временный каталог (после теста он удаляется — `addCleanup`), в нём файлы и каталоги.
- 25: пустая строка.
- 26–30: помощники: прочитать каталог через `fileops`, получить имена на диске.
- 31: пустая строка.
- 32–36: `rename` — весь путь переименования, как в программе: план → проверка → выполнение.
- 37: пустая строка.
- 38–41: `assert_error` — проверить, что функция выдаёт `FileError` именно с этим текстом.

- 42–43: пустые строки.

Строки 44–63. ListingTest — чтение каталога и пути

```
44  class ListingTest(DirectoryCase):
45      def test_folders_go_first_then_names_without_case(self):
46          names = [entry.name for entry in self.entries()]
47          self.assertEqual(names, ['a.dir', 'Фото', 'a.txt', 'b.md', 'b.txt', 'IMG_01.jpg', 'IMG_02.jpg', 'мой файл.txt'])
48
49      def test_missing_directory(self):
50          missing = os.path.join(self.directory, 'нет')
51          self.assert_error('Каталог не найден: ' + missing, fileops.list_directory, missing)
52
53      def test_resolve_relative_parent_and_absolute(self):
54          photos = fileops.resolve_directory(self.directory, 'фото')
55          self.assertEqual(photos, os.path.join(self.directory, 'Фото'))
56          self.assertEqual(fileops.resolve_directory(photos, '..'), self.directory)
57          self.assertEqual(fileops.resolve_directory(photos, self.directory), self.directory)
58
59      def test_resolve_errors(self):
60          self.assert_error('Каталог не найден: Нет', fileops.resolve_directory, self.directory, 'Нет')
61          self.assert_error('Это файл, а не каталог: a.txt', fileops.resolve_directory, self.directory, 'a.txt')
62
63
```

- 44–47: каталоги идут первыми, имена — без учёта регистра.
- 48: пустая строка.
- 49–51: несуществующий каталог — ошибка с нужным текстом.
- 52: пустая строка.
- 53–57: относительный путь (регистр не важен), `..` и абсолютный путь.
- 58: пустая строка.
- 59–61: ошибки: каталога нет; указан файл.
- 62–63: пустые строки.

Строки 64–88. MaskTest — маски и преобразования имён

```
64  class MaskTest(unittest.TestCase):
65      def test_matches(self):
66          self.assertTrue(fileops.matches('report.TXT', '*.txt'))
67          self.assertTrue(fileops.matches('a.txt', '?.txt'))
68          self.assertFalse(fileops.matches('ab.txt', '?.txt'))
69          self.assertTrue(fileops.matches('.txt', '*.txt'))
70          self.assertFalse(fileops.matches('a.txt.bak', '*.txt'))
71          self.assertTrue(fileops.matches('a[1].txt', 'a[1].*'))
72
73      def test_substitute(self):
74          self.assertEqual(fileops.substitute('a.txt', '*.txt', '*.md'), 'a.md')
75          self.assertEqual(fileops.substitute('IMG_01.jpg', 'IMG_*.jpg', 'отпуск_*.jpg'), 'отпуск_01.jpg')
76          self.assertEqual(fileops.substitute('a.b.txt', '*.*', '*.bak'), 'a.b.bak')
77          self.assertEqual(fileops.substitute('x1-report.txt', '?1-*.txt', '*_?.txt'), 'report_x.txt')
78          self.assertEqual(fileops.substitute('a.txt', '*.txt', 'все.txt'), 'все.txt')
79
80      def test_transforms(self):
81          self.assertEqual(fileops.add_prefix('a.txt', 'new_'), 'new_a.txt')
82          self.assertEqual(fileops.add_suffix('a.tar.gz', '_old'), 'a.tar_old.gz')
83          self.assertEqual(fileops.add_suffix('README', '_old'), 'README_old')
84          self.assertEqual(fileops.change_extension('a.txt', '.md'), 'a.md')
85          self.assertEqual(fileops.change_extension('README', 'md'), 'README.md')
86          self.assertEqual(fileops.change_extension('a.txt', ''), 'a')
87
88
```

- 64–71: совпадение масок: регистр не важен, `?` — ровно один символ, маска сравнивается с именем целиком, квадратные скобки в имени не ломают выражение.
- 72: пустая строка.
- 73–78: подстановка по новой маске, в том числе перестановка частей.
- 79: пустая строка.
- 80–86: три действия окна группы: в начало, в конец перед расширением (в том числе без расширения), смена расширения.
- 87–88: пустые строки.

Строки 89–152. RenameTest — переименование на диске

```
 89  class RenameTest(DirectoryCase):
 90      def test_single_file(self):
 91          self.assertEqual(self.rename('A.TXT', 'c.txt'), [Rename('a.txt', 'c.txt')])
 92          self.assertIn('c.txt', self.names())
 93          self.assertNotIn('a.txt', self.names())
 94
 95      def test_group_by_mask(self):
 96          changes = self.rename('IMG_*.jpg', 'отпуск_*.jpg')
 97          self.assertEqual(changes, [Rename('IMG_01.jpg', 'отпуск_01.jpg'), Rename('IMG_02.jpg', 'отпуск_02.jpg')])
 98          self.assertIn('отпуск_02.jpg', self.names())
 99
100      def test_mask_skips_folders(self):
101          self.assertEqual(self.rename('a.*', 'z.*'), [Rename('a.txt', 'z.txt')])
102          self.assertIn('a.dir', self.names())
103
104      def test_case_only_change(self):
105          self.rename('a.txt', 'A.TXT')
106          self.assertIn('A.TXT', self.names())
107
108      def test_errors_from_the_agreed_table(self):
109          self.assert_error('Файл не найден: нет.txt', self.rename, 'нет.txt', 'x.txt')
110          self.assert_error('Нет файлов, соответствующих маске: *.xyz', self.rename, '*.xyz', '*.abc')
111          self.assert_error('Нет файлов, соответствующих маске: Фот*', self.rename, 'Фот*', 'x*')
112          self.assert_error('Файл с именем b.txt уже существует. Переименование отменено.', self.rename, 'a.txt', 'b.txt')
113
114      def test_other_errors(self):
115          self.assert_error('Это каталог, а не файл: Фото', self.rename, 'фото', 'x')
116          self.assert_error('Каталог с именем фото уже существует. Переименование отменено.',
117                            self.rename, 'a.txt', 'фото')
118          self.assert_error('Новое имя совпадает со старым: a.txt', self.rename, 'a.txt', 'a.txt')
119          self.assert_error('В новой маске знаков * больше, чем в исходной: *_*.txt', self.rename, '*.txt', '*_*.txt')
120          self.assert_error('Знаки * и ? в новом имени допустимы только вместе с маской: *.md',
121                            self.rename, 'a.txt', '*.md')
122          self.assert_error('Недопустимое имя файла: a:b. Нельзя использовать символы \\ / : * ? " < > |',
123                            self.rename, 'a.txt', 'a:b')
124          self.assert_error('Недопустимое имя файла: x.. Имя не может заканчиваться точкой или пробелом.',
125                            self.rename, 'a.txt', 'x.')
126          self.assert_error('Недопустимое имя файла: con.txt. Это имя зарезервировано системой.',
127                            self.rename, 'a.txt', 'con.txt')
128          self.assert_error('Имя указывается без пути, команда работает в текущем каталоге: Фото\\a.txt',
129                            self.rename, 'Фото\\a.txt', 'b')
130
131      def test_conflict_cancels_the_whole_group(self):
132          before = self.names()
133          self.assert_error('Файл с именем b.md уже существует. Переименование отменено.', self.rename, '*.txt', '*.md')
134          self.assert_error('Файлы IMG_01.jpg и IMG_02.jpg получили бы одно имя фото.jpg. Переименование отменено.',
135                            self.rename, 'IMG_*.jpg', 'фото.jpg')
136          self.assertEqual(self.names(), before)
137
138      def test_failure_in_the_middle_restores_names(self):
139          before = self.names()
140          real_rename = os.rename
141
142          def failing_rename(source, target):
143              if os.path.basename(source) == 'IMG_02.jpg':
144                  raise PermissionError(13, 'Отказано в доступе')
145              real_rename(source, target)
146
147          with mock.patch('fileops.os.rename', failing_rename):
148              self.assert_error('Не удалось переименовать IMG_02.jpg: нет доступа или файл открыт в другой программе. '
149                                'Переименование отменено.', self.rename, 'IMG_*.jpg', 'x_*.jpg')
150          self.assertEqual(self.names(), before)
151
152
```

- 89–93: один файл: регистр в имени не важен, на диске появилось новое имя и исчезло старое.
- 94: пустая строка.
- 95–98: группа по маске.
- 99: пустая строка.
- 100–102: маска пропускает каталоги.
- 103: пустая строка.
- 104–106: смена только регистра.
- 107: пустая строка.
- 108–112: тексты ошибок из таблицы варианта.

- 113: пустая строка.
- 114–129: остальные ошибки: каталог вместо файла, занято каталогом, имя не изменилось, лишняя *, * без маски, запрещённый символ, точка в конце, зарезервированное имя, путь в имени.
- 130: пустая строка.
- 131–136: правило «всё или ничего»: при конфликте не меняется ни один файл.
- 137: пустая строка.
- 138–150: сбой посередине: `mock.patch` подменяет `os.rename` функцией, которая падает на втором файле; проверяется, что первый файл получил своё имя обратно.
- 151–152: пустые строки.

Строки 153–169. TextTest и запуск

```
153 class TextTest(unittest.TestCase):
154     def test_sizes(self):
155         self.assertEqual(fileops.format_size(0), '0 Б')
156         self.assertEqual(fileops.format_size(1023), '1023 Б')
157         self.assertEqual(fileops.format_size(1024), '1 КБ')
158         self.assertEqual(fileops.format_size(1536), '1,5 КБ')
159         self.assertEqual(fileops.format_size(5 * 1024 ** 2), '5 МБ')
160 
161     def test_counts(self):
162         self.assertEqual(fileops.renamed_text(1), 'Переименован 1 файл')
163         self.assertEqual(fileops.renamed_text(3), 'Переименовано 3 файла')
164         self.assertEqual(fileops.renamed_text(11), 'Переименовано 11 файлов')
165         self.assertEqual(fileops.renamed_text(21), 'Переименован 21 файл')
166 
167 
168 if __name__ == '__main__':
169     unittest.main()
```

- 153–159: размеры в байтах, КБ и МБ.
- 160: пустая строка.
- 161–165: формы слова «файл» после числа.
- 166–167: пустые строки.
- 168–169: запуск всех тестов при прямом запуске файла.

8. Запуск и проверка

```
cd lab3
python make_demo.py                создать или обновить папку demo
python nc_menu.py demo             запустить программу
python -m unittest test_fileops    запустить тесты
```

- Нужен только Python 3.10 или новее; ничего не компилируется и не устанавливается — все используемые модули стандартные.
- `demo` в конце команды обязательно, иначе программа откроет саму папку `lab3`.
- Кроме 17 тестов `fileops.py`, программа проверялась автоматическим сценарием (30 проверок): клавиши, щелчки, все окна, файлы на диске после каждого шага и то, что в окне нет ни одного виджета, кроме холста.